

Welcome Back

To those just joining now: this blog is part of my **DataEngineering101** series, where we break down essential tooling and workflows used in modern data pipelines. In the previous article, we built a basic data pipeline using Docker, Airflow, dbt, and Postgres – a batch-based pipeline that ran on a schedule and delivered data daily.

Now, we're moving beyond periodic batch jobs into **real-time, event-driven processing**. In this tutorial, we'll build a **streaming data pipeline** using APIs, Apache Kafka, Apache Spark, and AWS. If this sounds overwhelming, don't worry, we'll walk through the architecture one component at a time.

By the end, you'll understand how data can flow continuously from a source API, into Kafka, through Spark for processing, and into Amazon S3 for long-term storage, all while using Docker to simulate a realistic production-like environment locally. We'll also show how to use Airflow to orchestrate this system, making sure each component runs in the correct sequence.

Tools and Services We Will Be Using

Before diving into the actual pipeline, let's clarify what the pipeline is doing and introduce the main tools it consists of.

Pipeline Layers

Every modern data pipeline follows a rough pattern consisting of five core layers:

- **Ingestion** – Pulling data from external sources (e.g. APIs, logs, databases).
- **Transformation** – Cleaning, typing, and restructuring data into usable formats.
- **Storage** – Persisting raw or transformed data (e.g. Postgres, S3, Data Lake).
- **Orchestration** – Managing execution order, schedules, retries, and dependencies.
- **Visualization** – Delivering results to end-users via dashboards or reports.

In this project, we're skipping visualization. The final output is structured JSON stored in an S3 bucket. So we'll focus on:

- Ingestion
- Storage
- Transformation
- Orchestration

The Tech Stack

Apache Kafka – Event Ingestion and Streaming

Kafka is a **distributed event streaming platform**. "Distributed" means it can run across multiple machines to scale horizontally. "Event streaming" refers to its ability to handle continuous flows of data in real time.

At its core, Kafka facilitates communication between two roles:

- **Producers** – applications that send data into Kafka.
- **Consumers** – applications that read data from Kafka.

Kafka's job is to move data from producers to consumers **efficiently and durably**, with minimal transformation along the way.

Each piece of data sent into Kafka is called an **event** — a discrete record of something that happened (e.g., a new user signing up, a sensor reading, etc.). Kafka stores these events as **byte arrays** in **topics**. A *topic* is essentially a named stream, or channel, for a specific kind of event. For example, a topic called "user_signups" might hold one event per new user registration.

Writing Events to Kafka (Producer Side)

To send data to Kafka, we must first serialize it — i.e., convert it into a byte array format that Kafka can handle. If we take user signup example from earlier, a simplified example using Python would look like:

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(bootstrap_servers='localhost:9092',
                          value_serializer=lambda v: json.dumps(v).encode('utf-8'))

event_data = {"username": "jdoe", "timestamp": "2025-11-29T12:00:00Z"}
producer.send("user_signups", value=event_data)
producer.flush()
```

This snippet:

- Connects to a Kafka broker at localhost:9092.
- Serializes a dictionary (event_data) into a JSON byte string.
- Sends it to the topic "user_signups".

- Flushes the buffer to ensure the message is delivered.

Reading Events from Kafka (Consumer Side)

On the consumer side, the message needs to be **deserialized** — converted back from a byte string into a usable data structure (e.g., a Python dictionary or Spark DataFrame). A basic Python consumer would look like this:

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    "user_signups",
    bootstrap_servers='localhost:9092',
    auto_offset_reset='earliest',
    value_deserializer=lambda x: json.loads(x.decode('utf-8'))
)

for message in consumer:
    print("Consumed event:", message.value)
```

In Spark, deserialization happens as part of the read logic where you cast the Kafka value column to string and parse it with `from_json`.

Topics, Partitions, and Parallelism

So far, we've established that Kafka transmits data between **producers** and **consumers** via discrete units called **events**, and these events are grouped into **topics**. But topics are more than just labels — they also act as **append-only logs** and can be **partitioned** to achieve parallelism and scalability.

Let's break that down.

Internally, a **Kafka topic is implemented as an append-only log**.

A **log** is a time-ordered, immutable sequence of records. Once an event is written to the end of the log, it cannot be changed or deleted — it's stored exactly as it arrived, in the order it arrived.

This log-like structure is critical. It gives Kafka the ability to:

- Maintain event order (within partitions),
- Allow multiple consumers to read the same data independently,

- Replay historical data, and
- Handle failures by reprocessing from a known point (using offsets).

Kafka allows each topic to be split into **partitions**. A **partition** is simply a smaller, independently ordered subset of a topic. Each partition maintains its own log, its own ordering, and its own offset tracking.

Partitioning is what allows Kafka to scale. It enables:

- **Parallelism** – Multiple producers can write to different partitions of the same topic simultaneously. Likewise, multiple consumers in the same consumer group can read in parallel, each from a separate partition.
- **Load distribution/balancing** – Partitions are spread across different Kafka brokers (i.e., Kafka server nodes), preventing any single broker from becoming a bottleneck.
- **Horizontal scalability** – If more throughput is needed, you can add more partitions and scale your consumers accordingly.

Here's a mental model:

- An **event (X)** is a single data record — e.g., {"user_id": 123, "action": "click"}.
- A **topic (Y)** is a named stream of similar events — e.g., "user_activity".
- A **partition (Z)** is a distinct slice of that topic — e.g., "user_activity", partition 3.

So you can think of it like this:

Events (X) are written to a topic (Y), which is composed of one or more partitions (Z). Each partition is its own ordered log, enabling distributed, parallel processing.

A even more simplified analogy would be this:

Imagine you're baking multiple cakes.

- Each **ingredient** (e.g., sugar, flour, eggs) represents an **event** — a unit of data.
- A **cake** is a **topic** — a collection of related ingredients that form a whole.
- When you **slice** the cake into pieces, each **slice** is like a **partition** — a smaller, ordered segment of the whole, which can be handled and served independently.

Why does all of this matter?

Suppose you have a topic with only one partition. Only one consumer instance can read from it at a time. But if the topic has four partitions, four consumers (belonging to the same group) can each process a different partition concurrently — this dramatically increases throughput and reduces latency.

However, this comes with a tradeoff: **Kafka guarantees ordering only within a partition**, not across partitions. If the order of related events matters — say, all events tied to a single user — your producer must consistently route those events to the **same partition**. This is usually done by setting a **partition key**, like a user ID.

If that's not intuitive, think of it this way: partitions are like lanes on a highway. You can move more cars at once with more lanes, but cars in different lanes don't necessarily arrive in order. If you want things to arrive in sequence, you keep them in the same lane.

Finally, each **consumer tracks its own offset** — the position in the partition it last successfully read. This offset tracking enables:

- Reliable message delivery (even after crashes),
- Independent consumption progress across multiple consumers, and
- Replay capabilities for reprocessing data from an earlier point.

This architecture — with **decoupled producers and consumers**, **partitioned logs**, and **offset-based tracking** — is what gives Kafka its core strengths: **high throughput**, **fault tolerance**, and **scalability**.

So to relate this all back into our pipeline:

- The **producer** is a Python service that fetches data from a public API.
- The **consumer** is Apache Spark, which continuously reads from Kafka and transforms the incoming data.
- Kafka acts as the **buffer** between them, decoupling the ingestion and processing layers.

We're running Kafka in **KRaft mode**, a newer configuration that removes the need for ZooKeeper. Previously, ZooKeeper was used to manage metadata and broker coordination. KRaft (Kafka Raft Metadata mode) simplifies deployment by consolidating those responsibilities directly into the Kafka brokers. (I haven't directly defined what a broker is, but I will provide a basic overview of how a Kafka broker works later).

Apache Spark – Data Transformation

Apache Spark is a **distributed analytics engine** designed to process large-scale data across clusters of machines. While it's traditionally associated with batch workloads (processing large chunks of data at scheduled intervals), it also supports **Structured Streaming**, which allows us to process **real-time data streams** using the same DataFrame-based APIs familiar from batch processing.

In contrast to Kafka — which serializes data into byte arrays optimized for transport — Spark **converts raw input into structured DataFrames**. This enables transformation, filtering, aggregation, and more using a rich, SQL-like API.

Setting Up Spark: Cluster Model

To run Spark in our pipeline, we deploy a **Spark Standalone cluster**, which consists of:

- **One Master node** – responsible for **coordinating the cluster**, assigning work, and managing the scheduling of Spark applications (called jobs).
- **Multiple Worker nodes** – these execute the actual tasks and transformations assigned by the master.

Each time you run a Spark job, the master breaks it into **tasks**, distributes them across the workers, and collects the results. This distributed model is what gives Spark its scalability and speed.

Code Integration: Using PySpark

To interface with the Spark cluster in code, we use **PySpark** — the official Python API for Apache Spark. PySpark acts as the bridge between our Python application and the underlying distributed Spark engine.

In our case, PySpark is installed inside a Docker container that connects to the Spark master. When we launch a Spark job (e.g., from Airflow or the CLI), the Python script will communicate with the Spark cluster through PySpark.

What Is a Spark Job?

A **Spark job** is a unit of work submitted to the Spark cluster. It defines:

- The source of the data (e.g., Kafka),
- How the data should be interpreted and parsed (e.g., casting byte arrays into strings, parsing JSON),
- The transformations to apply (e.g., select, filter, groupBy, aggregate),
- The destination (e.g., writing out to S3).

Under the hood, Spark leverages **Hadoop-compatible file systems** (such as Amazon S3 or HDFS) for reading/writing data and uses its **DAG scheduler** to build an execution plan for

your job. (Hadoop is what will we use later to direct where we will be sending the data to the bucket).

To simplify:

1. You define your input (Kafka topic).
2. You convert that input into a **DataFrame** using PySpark.
3. You define a series of transformations.
4. Spark breaks the transformations into stages and tasks.
5. Those tasks are distributed across workers.
6. Output is written to your destination (in our case, S3 as JSON files).

Example: Reading from Kafka, Transforming, and Writing to S3

Here's a simplified PySpark job that reads a stream of JSON events from a Kafka topic, parses them, and writes them to Amazon S3:

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import from_json, col

from pyspark.sql.types import StructType, StringType, TimestampType
```

What's happening here:

- Spark reads the Kafka byte stream.
- It casts the data into strings, then parses it into structured columns using a schema.
- The resulting DataFrame is written continuously to an S3 bucket in append mode.
- A checkpoint location is used to track progress and ensure fault-tolerance.

This job runs **continuously**, processing micro-batches under the hood — meaning Spark collects new events every few seconds and processes them in small groups. This model offers the scalability of batch with near-real-time behavior.

S3 v2

This is where the processed data from the Spark job actually lands. Amazon S3 (Simple Storage Service) is an object storage system built for durability, scalability, and cost-efficiency.

In concrete terms:

- S3 stores **objects**, not traditional files. (What's the difference?)
- Each object lives in a **bucket** and is identified by a **key** (a unique string that behaves like a file path).
- From Spark's point of view, you access objects using URIs like:

s3a://my-bucket/path/to/object.json

Here, my-bucket is the bucket name and path/to/object.json is the object key. There are no real "folders"; the / separators are just part of the key.

S3 is part of Amazon Web Services (AWS). In this pipeline, S3 acts as our **data lake** — the final destination for transformed, structured data.

A **data lake** is a centralized storage layer that can hold both raw and processed data in its native format (JSON, CSV, Parquet, etc.) without forcing a rigid schema upfront. Later, tools like Athena or Redshift Spectrum can query that data directly from S3 using SQL.

Why Use S3?

We use S3 for several reasons:

- It **decouples storage from compute** — Spark does the work, then exits; the output persists independently in S3.
- It integrates with downstream analytics tools:
 - Amazon **Athena** can run SQL directly over the S3 objects.
 - Amazon **Redshift Spectrum**, Snowflake, or dbt can treat S3 as an external storage layer.
- It scales transparently with data volume and supports high write throughput.

For this demo, we connect to a real S3 bucket — you need:

- A bucket name (e.g. my-bucket), and
- Valid AWS credentials with permission to write to that bucket.

If you do not want to use AWS directly, you can swap S3 for **MinIO**, a local S3-compatible server. From Spark's point of view, it still talks to an "S3" endpoint via the same `s3a://` URLs; only the endpoint and credentials differ. A simple local setup is to run MinIO via Docker and configure it with an access key, secret key, and HTTP endpoint, as described in the MinIO documentation.

AWS Credentials and Configuration

Spark needs credentials to authenticate with S3. Common options include:

- **Environment variables** inside the Spark container:
`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and optionally `AWS_SESSION_TOKEN`.
- The standard AWS config/credentials files on the host:
`~/.aws/credentials` and `~/.aws/config` (used if Spark runs outside Docker or if those files are mounted into the container).
- Explicit configuration in Spark's Hadoop options (what this project uses).

In this setup, credentials and S3 configuration are injected into the **Spark Docker container** via environment variables (for example from `.env.aws`), and the Spark session is initialized with those values.

Connecting Spark to S3

The key step is configuring Spark's Hadoop layer so that `s3a://...` URLs resolve correctly. The configuration includes:

- AWS region (e.g. `eu-west-1`),
- Access key, secret key, and optional session token,
- The S3 endpoint (for MinIO or custom S3-compatible services),
- Whether to use SSL and path-style access.

Example (conceptual) initialization:

```
spark = initialize_spark_session(  
    app_name,  
    region=s3_region,  
    access_key=access_key,
```

```

secret_key=secret_key,
session_token=session_token,
endpoint=endpoint_host, # e.g. s3.amazonaws.com or http://minio:9000
path_style=path_style, # True for MinIO, usually False for AWS
ssl_enabled=ssl_pref,
)

```

This is where the AWS/MinIO details are wired in. Once this session is created, any `s3a://bucket-name/...` path used in reads or writes will go to the configured storage backend.

Example Write Logic

Once the data is transformed in Spark, writing to S3 is straightforward:

```

parsed_df.writeStream \
    .format("json") \
    .option("path", "s3a://my-bucket/output/") \
    .option("checkpointLocation", "s3a://my-bucket/checkpoints/") \
    .outputMode("append") \
    .start() \
    .awaitTermination()

```

Key points:

- The path is an **S3A URI**:
`s3a://my-bucket/output/`
 Under the hood, Spark will create objects with keys like `output/part-0000-...json`.
- Data is written incrementally as **JSON files** in append mode. Each micro-batch from the Kafka stream becomes one or more new objects in `output/`.
- The `checkpointLocation` is **not optional** for structured streaming.
 - Spark's `.writeStream` requires a checkpoint directory so it can store:
 - Offsets it has already processed,
 - Batch IDs,

- State needed for exactly-once/at-least-once semantics (depending on the sink).
- On restart, Spark reads the checkpoint data to **reconstruct its state** and continue from where it left off instead of reprocessing the entire stream.

In this project, both the output data and the checkpoint metadata are stored in S3 (output/ and checkpoints/ prefixes within the same bucket). Later, in the Athena section, we will query the output/ prefix using SQL to verify that the stream has landed correctly in S3.

Apache Airflow

Apache Airflow is a **workflow scheduler** widely used in data engineering to define, monitor, and manage pipelines. Each workflow is expressed as a **Directed Acyclic Graph (DAG)** — a set of tasks that run in a specific order without cycles or repetition.

Even though our pipeline is designed for streaming data, we use Airflow to run each segment in **finite, repeatable bursts**. This approach gives us control over execution while preserving key benefits of streaming behavior.

Using Airflow allows us to:

- **Schedule regular ingestion and processing windows** (e.g., run every few minutes).
- **Handle transient failures** with automatic retries and alerting.
- **Track execution history** through logs and Airflow's built-in web interface.

To orchestrate containerized components like Spark, we use the **DockerOperator**, which runs specific commands inside Docker containers. For lightweight checks and logic, such as verifying that Kafka is reachable or confirming S3 credentials are valid, we use the **PythonOperator**.

Because this is a **streaming pipeline**, there aren't many complex DAG tasks — we're not chaining dozens of transformations. Instead, we use **checkpoint tasks** strategically:

- To validate that each core service (Kafka, Spark, S3) is up and reachable.
- To ensure the pipeline is wired correctly end to end before any processing begins.

This hybrid approach — using Airflow for orchestration, even in a streaming context — keeps things modular, transparent, and recoverable without compromising the nature of continuous data flow.

How This Would Differ in Actual Production vs Local Setup

In our local environment, we're running **Kafka**, **Spark**, **Airflow**, and other components using **Docker Compose** on a single machine. This setup simulates a distributed architecture, but in reality, it's all containerized and runs on your local system.

In a real production environment, each of these components would typically be replaced with **managed cloud services** for better scalability, availability, and operational simplicity.

Component	Local	Production
Kafka	Kafka container (KRaft)	Amazon MSK or Amazon Kinesis
Spark	Spark Standalone in Docker	AWS EMR, AWS Glue Streaming
Airflow	Airflow containerized with Docker	Amazon MWAA or AWS Step Functions
S3	Real S3 Bucket (or MinIO)	Same
Orchestration	Airflow DAGs	Airflow, Step Functions, or K8s Operators

Why This Still Matters

While the tooling might change in production, **the architecture and data flow stay the same**. Understanding how each component fits into the pipeline — where data is ingested, how it's processed, and where it's stored — gives you the flexibility to **swap out components** without changing the overall logic.

Once you're clear on the flow and boundaries between stages (Kafka → Spark → S3, orchestrated by Airflow), transitioning to production is mostly a matter of configuration. You're not rewriting your entire system — you're **replacing self-managed services with cloud-managed ones**.

The design decisions we've made here — separating ingestion, transformation, and storage, and orchestrating everything cleanly — make it easy to scale individual parts of the pipeline independently and move each layer to its production-grade equivalent.

Now that we have a solid understanding of the tools and their roles, we can start walking through each section of the system in detail — starting with how the **Spark cluster** is deployed and orchestrated.

Pipeline Overview

Revisiting the Pipeline Architecture

Now that we’ve walked through the core technologies individually, let’s circle back to the **overall data pipeline structure**.

As outlined earlier, a robust data pipeline is typically composed of five key layers:

- 1. **Ingestion** – pulling data in from sources like APIs, logs, or databases.
- 2. **Storage** – placing that data into a system where it can be managed and queried (e.g., Postgres, S3, data lakes).
- 3. **Transformation** – cleaning, typing, and reshaping it so it’s structured and consistent.
- 4. **Orchestration** – automating the flow, managing task order, failures, and retries.
- 5. **Visualization** – exposing data to end-users via dashboards or reports.

In our example, we **don’t focus on visualization**. The final output is a stream of JSON records written to S3, ready for consumption by downstream tools — but the reporting layer is intentionally omitted to keep the scope focused.

So, for our purposes, the active layers are:

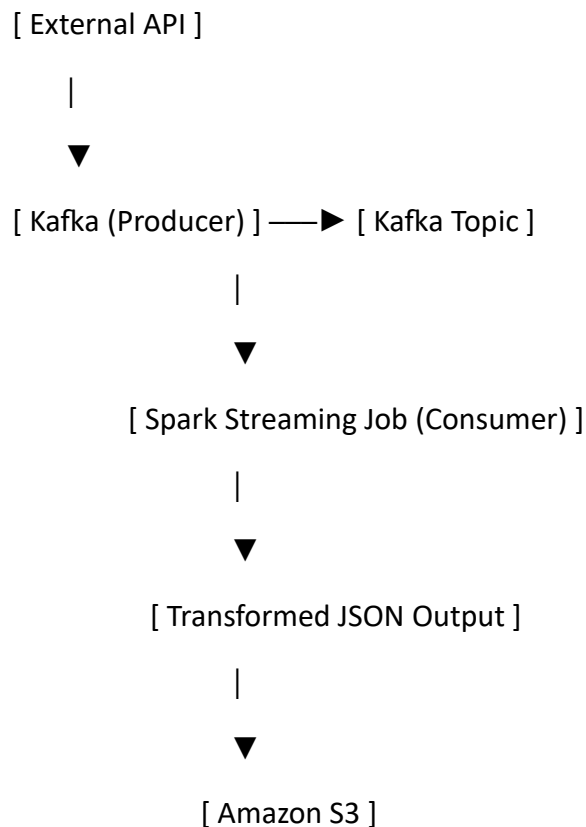
- **Ingestion**
- **Storage**
- **Transformation**
- **Orchestration**

Here’s how each part of our system maps to these layers:

Layer	Component(s)	Description
Ingestion	Custom Python API + Kafka	The producer fetches external data and pushes it into Kafka.
Transformation	Apache Spark + Kafka	Spark reads the stream from Kafka, parses and filters it, and prepares structured output.
Storage	Amazon S3 (or MinIO for local)	Transformed JSON records are persisted in an S3 bucket.
Orchestration	Apache Airflow (DAGs, DockerOperator, checkpoints)	Airflow coordinates the producer, Spark job, and health checks.

Diagrammatic View (Suggested Structure)

You could visualize the pipeline like this:



With **Airflow** sitting above this entire system, managing:

- When the producer runs,
- When the Spark job launches,
- Whether each component is alive and connected.

This structure gives you a **modular, testable, production-aware pipeline** that separates concerns cleanly — a foundational skill for building anything at scale.

Docker Compose

Docker Compose

Each part of this pipeline runs in its own Docker container: Kafka, Spark master and workers, Airflow webserver and scheduler, the Postgres metadata database, and so on. Docker gives us isolated environments; Docker Compose is what wires them together into a single stack.

All of the services are defined in a single docker-compose.yml file. The file is quite long, but conceptually it is split into three groups:

- **Airflow services** – Airflow webserver, scheduler, and the Postgres metadata database.
- **Kafka services** – Kafka broker (KRaft mode), Kafka UI, Kafka Connect, and Schema Registry.
- **Spark services** – Spark master and one or more Spark workers, plus a short-lived container for submitting the streaming job.

Some services use standard images (for example postgres:16 for the Airflow metadata DB). Others use **custom images** built from the Dockerfiles in this project (for example custom-airflow and custom-spark), so that the right Python packages and Spark connectors are available inside the containers.

Docker Compose also defines:

- The **shared networks** that let containers talk to each other by name (e.g. kafka-broker, spark-master, airflow-webserver), and
- The **volumes** that persist state across restarts (for example, PostgreSQL data and Airflow logs).

We will not inline the entire docker-compose.yml here, but the rest of this post walks through it piece by piece:

- In the **Kafka section**, we look at the broker, controller quorum, UI, Connect, and Schema Registry services.
- In the **Spark section**, we cover the Spark master, workers, and the streaming driver container that runs spark-submit.
- In the **Airflow section**, we explain how the webserver, scheduler, and metadata database are wired together.

Once you have the docker-compose.yml file and the accompanying Dockerfiles in place, you bring the entire stack up with a single command from the project root:

```
docker compose up -d
```

Compose will build the custom images if needed, start all containers, attach them to the correct networks, and leave the stack running in the background. From there, Kafka, Spark, and Airflow are all available for the rest of the pipeline.

Kafka Section

We'll start with the **Kafka** part of the stack, since everything else builds on it.

Kafka is the event backbone of this stack: everything else (Spark, Airflow, S3) is built on top of a stream of messages moving through Kafka topics.

Earlier we covered the basic data model:

- **Topics** – named streams of events.
- **Partitions** – shards of a topic for parallelism.
- **Producers** – write events to topics.
- **Consumers** – read events from topics.

To understand how Kafka actually runs in our Docker setup, we need to add a few more ideas:

- **Brokers**
- **Controllers**
- **Quorums (KRaft)**
- **Consumer groups**
- **Serialization**

Once those are clear, we can map them directly onto the Docker services.

Brokers, Controllers, Quorums, and Consumer Groups

Broker

A **Kafka broker** is just a single Kafka server process. It:

- Stores topic partitions on disk,
- Accepts messages from producers,
- Serves data to consumers,
- Replicates data to other brokers (in multi-broker clusters).

All of your topics “live” on brokers. In this project we run one broker, so all partitions sit on that single node.

Controller and quorum (KRaft mode)

Kafka also needs a component to coordinate cluster metadata:

- Which brokers are alive,
- Which broker is leader for each partition,

- Where topics and partitions are placed.

In modern Kafka, this is handled by a **controller** backed by a **metadata quorum**. Kafka calls this **KRaft** (Kafka + Raft). The quorum is a small set of controller nodes that run the Raft consensus algorithm to agree on metadata changes, so the cluster has a single source of truth.

In older deployments ZooKeeper did this job. In our setup we use a **single-node KRaft cluster** for simplicity: one process acts as both **broker** and **controller**.

That is why the environment variables look like this:

- KAFKA_PROCESS_ROLES=broker,controller
- KAFKA_NODE_ID=1
- KAFKA_CONTROLLER_QUORUM_VOTERS=1@kafka:29093

This means:

- The kafka container is **both** broker and controller.
- Its node ID is 1.
- The controller quorum consists of just this node at kafka:29093.

In production you would run multiple brokers and multiple controllers, but the logical structure is the same.

Consumer groups

The last core concept is the **consumer group**. A consumer group is a set of consumers that share the work of reading from a topic:

- Each partition of a topic is assigned to **one** consumer in the group.
- Different consumers can read different partitions **in parallel**.
- From Kafka's point of view, a consumer group represents one logical "application".

In this stack, Spark Structured Streaming acts as a **single consumer group**:

- The streaming job subscribes to a topic as one group,
- Spark spins up multiple tasks to read from different partitions concurrently,
- Offsets are tracked per group so the job can restart without losing its place.

Serialization: turning data into bytes

Kafka does not understand JSON, dicts, or objects. It only transports **byte arrays**.

That means every producer must:

1. Take an in-memory object (e.g. a Python dict),
2. **Serialize** it into bytes (for us, JSON-encode + UTF-8),
3. Send those bytes to Kafka.

On the consumer side we do the inverse:

1. Read bytes from Kafka,
2. **Deserialize** them (UTF-8 decode + json.loads),
3. Work with the resulting Python object.

So whenever you see JSON in the producer/consumer code, remember: Kafka itself is blind to JSON. Serialization is what makes the bridge between “Python objects” and “Kafka bytes”.

Topic creation from code

In this project you do **not** manually pre-create topics via CLI. The Python producer uses Kafka’s **Admin API** to ensure the topic exists before it starts publishing:

- It connects to the cluster as an admin client,
- Checks whether the topic is present,
- Creates it on-the-fly if needed (with a given partition count).

This is helpful in local/dev setups: you can spin up the stack, run the producer, and have the required topic(s) created automatically.

How our Python producer uses Kafka

Conceptually, the producer service in this stack does the following:

1. **Call an external API** using requests.get(...) to fetch JSON data.
2. **Normalize/flatten** the response into a clean Python dict.
3. **Serialize** that dict to JSON bytes.
4. **Publish** the event to a Kafka topic using the Kafka client.
5. Loop on a schedule, sending a new event batch every N seconds.

The details of this live in kafka_streaming_service.py, but the mapping is simple:

- External world → HTTP API → Python → Kafka topic.

- Later, Spark reads from that topic as a streaming source.

This closes the gap between the abstract “producer/consumer” discussion and the concrete code you actually run.

Kafka Services in Our Docker Compose

In the Docker Compose file, the Kafka layer is split into four services:

1. kafka – Kafka broker (in KRaft mode)
2. kafka_connect – Kafka Connect
3. kafka_schema_registry – Schema Registry
4. kafka_ui – Kafka UI

Each has a distinct role in a realistic Kafka ecosystem.

1. kafka – The Broker (KRaft)

kafka:

image: confluentinc/cp-kafka:7.6.1

container_name: kafka

volumes:

- kafka_data:/var/lib/kafka/data

environment:

- KAFKA_PROCESS_ROLES=broker,controller

- KAFKA_NODE_ID=1

- KAFKA_CONTROLLER_QUORUM_VOTERS=1@kafka:29093

- KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER

-

KAFKA_LISTENERS=INTERNAL://0.0.0.0:19092,EXTERNAL://0.0.0.0:9092,CONTROLLER://0.0.0.0:29093

-

KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT

- KAFKA_INTER_BROKER_LISTENER_NAME=INTERNAL

-

KAFKA_ADVERTISED_LISTENERS=INTERNAL://kafka:19092,EXTERNAL://\${DOCKER_HOST_IP:-localhost}:9092

- KAFKA_CLUSTER_ID=\${KAFKA_CLUSTER_ID}

- CLUSTER_ID=\${KAFKA_CLUSTER_ID}

- KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1

- KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=1

- KAFKA_TRANSACTION_STATE_LOG_MIN_ISR=1

- KAFKA_LOG_DIRS=/var/lib/kafka/data

Key points:

Persistent storage

volumes:

- kafka_data:/var/lib/kafka/data

- /var/lib/kafka/data is where Kafka stores its log segments (your topic partitions).
- The named volume kafka_data means the data will survive container restarts.

Roles and identity

- KAFKA_PROCESS_ROLES=broker,controller → this node is both broker and controller.
- KAFKA_NODE_ID=1 → unique numeric ID of this node.
- KAFKA_CONTROLLER_QUORUM_VOTERS=1@kafka:29093 → the metadata quorum consists of node 1 reachable at kafka:29093.

Listeners

-

KAFKA_LISTENERS=INTERNAL://0.0.0.0:19092,EXTERNAL://0.0.0.0:9092,CONTROLLER://0.0.0.0:29093

-

KAFKA_ADVERTISED_LISTENERS=INTERNAL://kafka:19092,EXTERNAL://\${DOCKER_HOST_IP:-localhost}:9092

- INTERNAL – 0.0.0.0:19092: used by **other containers** on the Docker network (Spark, Airflow, Kafka Connect, Kafka UI).

- EXTERNAL – 0.0.0.0:9092: exposed to your **host machine** on port 9092.
- CONTROLLER – 0.0.0.0:29093: used internally for controller quorum traffic.

KAFKA_ADVERTISED_LISTENERS is what clients actually see:

- Inside Docker: INTERNAL://kafka:19092.
- From the host: EXTERNAL://localhost:9092 (or \${DOCKER_HOST_IP} if set).

Replication factors

- KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1
- KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=1

These are set to 1 because we only have one broker. In a real cluster you would increase these to gain fault-tolerance.

This kafka service is what both:

- The **Python producer** (kafka_streaming_service.py), and
- The **Spark streaming job**

talk to as kafka:19092 from inside Docker.

2. kafka_connect – Kafka Connect

Kafka Connect is a framework for running managed **source** and **sink** connectors (e.g. database → Kafka, Kafka → S3).

kafka_connect:

image: confluentinc/cp-kafka-connect:7.6.1

depends_on:

- kafka

environment:

- CONNECT_BOOTSTRAP_SERVERS=kafka:19092

- CONNECT_GROUP_ID=connect-cluster

- CONNECT_CONFIG_STORAGE_TOPIC=_connect-configs

- CONNECT_OFFSET_STORAGE_TOPIC=_connect-offsets

- CONNECT_STATUS_STORAGE_TOPIC=_connect-status

- CONNECT_KEY_CONVERTER=org.apache.kafka.connect.json.JsonConverter

- CONNECT_VALUE_CONVERTER=org.apache.kafka.connect.json.JsonConverter
- CONNECT_PLUGIN_PATH=/usr/share/java,/etc/kafka-connect/jars

Key points:

- depends_on: - kafka → Connect starts **after** the broker is healthy.
- CONNECT_BOOTSTRAP_SERVERS=kafka:19092 → Connect uses the **internal** listener.
- _connect-configs, _connect-offsets, _connect-status are internal topics for its own state.
- JSON converters mean keys/values are treated as JSON by default.
- CONNECT_PLUGIN_PATH is where you can drop extra connector JARs.

This stack does not yet rely on Connect for the main pipeline (Spark handles the S3 sink), but wiring it in makes it trivial to add, for example, a JDBC source connector later.

3. kafka_schema_registry – Schema Registry

Schema Registry manages schemas for Kafka messages (Avro, Protobuf, JSON Schema) and allows safe schema evolution.

kafka_schema_registry:

image: confluentinc/cp-schema-registry:7.6.1

depends_on:

- kafka

environment:

- SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS=PLAINTEXT://kafka:19092
- SCHEMA_REGISTRY_LISTENERS=http://0.0.0.0:8081
- SCHEMA_REGISTRY_KAFKASTORE_TOPIC_REPLICATION_FACTOR=1
 - Connects to Kafka via kafka:19092.
 - Exposes an HTTP API on port 8081.
 - Internal metadata topic replication is also 1 (single-broker setup).

In this project we send plain JSON without schema enforcement, but Schema Registry is present to mirror a realistic Kafka deployment and to allow you to migrate to Avro/Protobuf later.

4. kafka_ui – Kafka UI

kafka_ui (Provectus Kafka UI) is a web interface for inspecting the cluster:

- View topics and partitions,
- Browse messages,
- Inspect consumer groups,
- See Connect and Schema Registry wiring.

kafka_ui:

image: provectuslabs/kafka-ui:v0.7.2

depends_on:

- kafka
- kafka_schema_registry
- kafka_connect

environment:

- KAFKA_CLUSTERS_0_NAME=dev-cluster
- KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:19092
- KAFKA_CLUSTERS_0_SCHEMAREGISTRY=http://kafka_schema_registry:8081
- KAFKA_CLUSTERS_0_KAFKACONNECT_0_ADDRESS=http://kafka_connect:8083

You access it at <http://localhost:8888> (as mapped in the compose file).

Kafka UI ties together:

- The broker,
- Schema Registry,
- Kafka Connect,

which makes it much easier to debug topics, messages, consumer groups, and offsets.

Networks: kafka_network vs docker_streaming

All Kafka-related services attach to two Docker networks:

networks:

kafka_network:

driver: bridge

docker_streaming:

driver: bridge

Pattern:

- **kafka_network** – Kafka ↔ Spark internal traffic.
 - kafka, kafka_connect, kafka_schema_registry, kafka_ui
 - spark-master, spark_worker_*, spark_streaming

This lets Spark streaming jobs talk to kafka:19092 cleanly.

- **docker_streaming** – Cross-service orchestration: Kafka ↔ Spark ↔ Airflow.
 - Airflow tasks (health checks, DAG-triggered jobs) can call Kafka and Spark.
 - Kafka services are reachable from Airflow containers using the same internal hostnames.

So:

- Kafka's **internal** wiring to Spark → kafka_network.
 - Full pipeline orchestration (Airflow scheduling Spark jobs that read from Kafka and write to S3) → docker_streaming.
-

At this point you have a direct mapping from Kafka concepts:

- Broker, controller, quorum, consumer groups, serialization, topics

to the concrete Docker services, environment variables, and networks that actually run Kafka in this project.

Kafka Code

Kafka Code

Now that we've wired up Kafka at the Docker level, we can look at the actual producer service: kafka_streaming_service.py.

The core idea is simple:

Take data from an external API → normalize it → serialize it to JSON bytes → send it into a Kafka topic for Spark to consume.

To do that cleanly, the script is structured into four main stages:

1. Setup (Kafka client + topic creation)
2. Ingest (HTTP call to the API)
3. Transform (flatten and clean the payload)
4. Publish (serialize and send events to Kafka in a loop)

The code already follows that structure. Below is how each part fits together.

Imports, Type Hints, and Dynamic Kafka Import

At the top of the file, we handle imports and type hints:

```
if TYPE_CHECKING: # Only for type-checkers.
```

```
    from confluent_kafka import Producer
```

```
    from confluent_kafka.admin import AdminClient, NewTopic
```

```
else:
```

```
    Producer = Any
```

```
def _require_confluent_kafka() -> tuple[Any, Any, Any]:
```

```
    """Import confluent_kafka and raise a clear error if missing."""
```

```
    try:
```

```
        from confluent_kafka import Producer as _Producer
```

```
        from confluent_kafka.admin import AdminClient as _AdminClient, NewTopic as  
_NewTopic
```

```
    except ModuleNotFoundError as exc:
```

```
        raise ModuleNotFoundError(
```

```
            "confluent_kafka is required. Install it where this script runs (e.g., `pip install  
confluent-kafka`)."

```

```
        ) from exc
```

```
return _Producer, _AdminClient, _NewTopic
```

What this is doing:

- `TYPE_CHECKING` is a flag used by static type-checkers like mypy.
 - When `TYPE_CHECKING` is `True`, we import the real `Producer`, `AdminClient`, and `NewTopic` so the type checker knows the proper types.
 - At runtime (`TYPE_CHECKING` is `False`), we avoid importing them here and just alias `Producer = Any`.
- `_require_confluent_kafka()` is a helper that:
 - Tries to import `Producer`, `AdminClient`, and `NewTopic` from `confluent_kafka` at runtime.
 - Raises a clear error if `confluent-kafka` is not installed.

Under the hood, everywhere you see `_require_confluent_kafka()` being used, you should read it as “give me the real `confluent_kafka` `Producer` and `admin` types at runtime, or fail loudly if the library is missing”.

We then define a few constants:

Connection targets:

- Inside Docker network → "kafka:19092"

- From host OS → "localhost:9092"

KAFKA_BOOTSTRAP = "kafka:19092"

KAFKA_BOOTSTRAP = "localhost:9092"

API_ENDPOINT = "https://randomuser.me/api/?results=1"

KAFKA_TOPIC = "names_topic"

PAUSE_INTERVAL = 10

STREAMING_DURATION = 120

- `KAFKA_BOOTSTRAP` is the address of the Kafka broker **inside** the Docker network (`kafka:19092`).
- `API_ENDPOINT` is the external API.
- `KAFKA_TOPIC` is the topic we'll publish user events to.

- PAUSE_INTERVAL and STREAMING_DURATION control how often we send events and for how long.
-

Stage 1 – Setup (Ensure Kafka Topic Exists)

```
def ensure_topic(
    admin_bootstrap: str,
    topic: str,
    num_partitions: int = 1,
    replication_factor: int = 1,
) -> None:
    """Create the topic if missing; no-op if it already exists."""
    _, AdminClient, NewTopic = _require_confluent_kafka()
    admin = AdminClient({"bootstrap.servers": admin_bootstrap})

    md = admin.list_topics(timeout=5)
    if topic in md.topics:
        return

    futures = admin.create_topics(
        [NewTopic(topic, num_partitions=num_partitions,
        replication_factor=replication_factor)]
    )
    for t, f in futures.items():
        try:
            f.result()
            print(f"Created topic: {t}")
        except Exception as e:
            print(f"Topic create warning for {t}: {e}")
```

This function:

- Connects to Kafka's **Admin API** (AdminClient from confluent_kafka).
- Checks if the topic already exists.
- If not, it creates it with:
 - num_partitions (here 1, but easily adjustable),
 - replication_factor (1 in our single-broker demo).

We will call ensure_topic(...) from the main streaming loop. You can mentally read that call as:

“Before streaming, create the Kafka topic if it does not already exist.”

That means you can start the producer without manually creating the topic with CLI tools.

Stage 2 – Ingest (HTTP Call to API)

```
def retrieve_user_data(
```

```
    url: str = API_ENDPOINT,
```

```
    retries: int = 3,
```

```
    timeout: int = 10,
```

```
) -> Dict[str, Any]:
```

```
    """GET one random user; retry a few times; fall back to synthetic on repeated failure."""
```

```
    last_exception: Exception | None = None
```

```
    for attempt in range(1, retries + 1):
```

```
        try:
```

```
            resp = requests.get(url, timeout=timeout)
```

```
            resp.raise_for_status()
```

```
            data = resp.json()
```

```
            return data["results"][0]
```

```
        except Exception as exc:
```

```
            last_exception = exc
```

```
            if attempt == retries:
```

```
                LOGGER.warning(
```

```

        "Failed to reach %s after %s attempts; using fallback.",
        url,
        retries,
        exc_info=exc,
    )
    break
    time.sleep(1)

```

if last_exception is not None:

```
    return _fallback_user_data()
```

```
    raise RuntimeError("retrieve_user_data exited without data or exception")
```

This function:

- Calls the randomuser.me API and returns a single user object from data["results"][0].
- Retries up to retries times on failure.
- Logs a warning and falls back to _fallback_user_data() if it cannot reach the API (so the stream keeps running even if the API is flaky).

This is the ingestion step: raw data in, straight from HTTP.

Stage 3 – Transform (Normalize API Payload)

We define a couple of helpers:

```
def safe_float(x: Any) -> Optional[float]:
```

```
    """float(x) or None on failure."""
```

```
    try:
```

```
        return float(x)
```

```
    except Exception:
```

```
        return None
```

```
def encrypt_zip(zip_code: Any) -> int:
```

```
    """Hash postcode → int (deterministic pseudonym)."""
```

```
    md5_hex = hashlib.md5(str(zip_code).encode("utf-8")).hexdigest()
```

```
    return int(md5_hex, 16)
```

```
    # For stronger privacy, prefer HMAC-SHA256 with a secret.
```

- `safe_float` safely casts a value to float, returning None if it fails.
- `encrypt_zip` hashes the postcode into a large integer, so we do not expose raw postcodes directly (pseudo-anonymisation).

Then the main transformation:

```
def transform_user_data(d: Dict[str, Any]) -> Dict[str, Any]:
```

```
    """Flatten/normalize API payload; cast coords; hash postcode."""
```

```
    title = d["name"]["title"]
```

```
    first = d["name"]["first"]
```

```
    last = d["name"]["last"]
```

```
    name = f"{title}. {first} {last}"
```

```
    address = f"{d['location']['street']['number']}, {d['location']['street']['name']}"
```

```
    city = d["location"]["city"]
```

```
    nation = d["location"]["country"]
```

```
    postcode_hashed = str(encrypt_zip(d["location"]["postcode"]))
```

```
    lat = safe_float(d["location"]["coordinates"]["latitude"])
```

```
    lon = safe_float(d["location"]["coordinates"]["longitude"])
```

```
    email = d.get("email")
```

```
    return {
```

```
        "name": name,
```

```
        "gender": d.get("gender"),
```

```
        "address": address,
```

```
"city": city,
"nation": nation,
"zip": postcode_hashed,
"latitude": lat,
"longitude": lon,
"email": email,
}
```

This step:

- Builds a clean name field from title, first, and last.
- Flattens nested address fields into:
 - address, city, nation, zip.
- Hashes the postcode for privacy.
- Parses the latitude/longitude as floats.
- Returns a normalised dictionary with a stable schema — the exact schema Spark will later parse from Kafka.

At this point we have **clean, structured events**: a consistent JSON shape that is safe to store and process. The next stage is to take these normalised events and stream them into Kafka.

Stage 4 – Publish (Build Producer and Send Events)

First we build a Producer:

def build_producer(bootstrap: str) -> Producer:

```
"""Build a Producer with idempotence, acks=all, light batching, and compression."""
conf = {
    "bootstrap.servers": bootstrap,
    "client.id": "producer_instance",
    "acks": "all",
    "enable.idempotence": True,
    "compression.type": "gzip",
    "linger.ms": 20,
```

```

    "batch.size": 64_000,
}

ProducerCls, _, _ = _require_confluent_kafka()

return ProducerCls(conf)

```

Here:

- `_require_confluent_kafka()` gives us the real `confluent_kafka.Producer` class at runtime (`ProducerCls`).
- We then instantiate it with a configuration tuned for a small demo stream.

Config highlights:

- `bootstrap.servers` → which Kafka cluster to talk to (`kafka:19092` in Docker).
- `acks="all"` → the leader waits for all in-sync replicas to acknowledge (stronger durability).
- `enable.idempotence=True` → avoids duplicate messages in certain failure scenarios.
- `compression.type="gzip"` → compresses data on the wire.
- `linger.ms` and `batch.size` → control batching behaviour for better throughput.

Delivery callback:

```

def delivery_status(err, msg):
    """Per-message delivery report."""

    if err is not None:
        print("Delivery failed:", err)
    else:
        print(f"Delivered to {msg.topic()} [{msg.partition()}] @ offset {msg.offset()}")

```

This is just logging whether each message made it to Kafka.

Publishing a single record:

```

def publish_once(producer: Producer, topic: str, data: Dict[str, Any]) -> None:
    """Serialize dict → JSON bytes and enqueue to Kafka."""

    value = json.dumps(data).encode("utf-8")

    LOGGER.info("Producing record to topic=%s payload=%s", topic, data)

    producer.produce(

```



```

    topic,
    value=value,
    on_delivery=delivery_status,
)
producer.poll(0) # Drive I/O and callbacks.

```

Here we:

- Convert the Python dict into JSON, then into bytes (.encode("utf-8")).
- Call producer.produce(...) with our topic and value.
- Call producer.poll(0) to allow the client to send data and handle callbacks.

This is where the “convert to byte arrays and send as events” actually happens. The only thing that reaches Kafka is a stream of JSON byte payloads.

Orchestrator – Putting It All Together

Finally, the orchestrator ties all four stages into a streaming loop:

def initiate_stream() -> None:

```

"""Run: ensure topic → build producer → loop(fetch→transform→produce) → flush."""

```

```

LOGGER.info(
    "Starting Kafka stream: bootstrap=%s topic=%s interval=%s duration=%s",
    KAFKA_BOOTSTRAP,
    KAFKA_TOPIC,
    PAUSE_INTERVAL,
    STREAMING_DURATION,
)

```

Stage 1: make sure the Kafka topic exists before we start producing.

This uses confluent_kafka's AdminClient under the hood.

```

ensure_topic(KAFKA_BOOTSTRAP, KAFKA_TOPIC, num_partitions=1, replication_factor=1)

```

```
# Stage 4 (part 1): build a confluent_kafka Producer instance.
```

```
producer = build_producer(KAFKA_BOOTSTRAP)
```

```
iterations = STREAMING_DURATION // PAUSE_INTERVAL
```

```
try:
```

```
    for i in range(iterations):
```

```
        # Stage 2: ingest from HTTP
```

```
        raw = retrieve_user_data()
```

```
        # Stage 3: transform into a clean, structured event
```

```
        payload = transform_user_data(raw)
```

```
        LOGGER.info("Iteration %s/%s – sending payload %s", i + 1, iterations, payload)
```

```
        # Stage 4 (part 2): publish to Kafka
```

```
        publish_once(producer, KAFKA_TOPIC, payload)
```

```
        time.sleep(PAUSE_INTERVAL)
```

```
finally:
```

```
    remaining = producer.flush(10)
```

```
    if remaining:
```

```
        LOGGER.warning("Flush timed out with %s message(s) still in queue.", remaining)
```

```
    else:
```

```
        LOGGER.info("Kafka producer flushed all pending messages.")
```

```
if __name__ == "__main__":
```

```
    initiate_stream()
```

```
Flow:
```

1. **Ensure topic exists** (ensure_topic): create the Kafka topic if it does not already exist, via `confluent_kafka.AdminClient`.
2. **Build a producer** (build_producer): create a `confluent_kafka.Producer` instance configured for reliable streaming.
3. Compute how many iterations fit into `STREAMING_DURATION`.
4. For each iteration:
 - Call the API (`retrieve_user_data`),
 - Transform the payload to a stable schema (`transform_user_data`),
 - Publish it to Kafka (`publish_once`),
 - Sleep for `PAUSE_INTERVAL` seconds.
5. On shutdown, `flush()` the producer to make sure all messages are sent.

From Kafka's perspective this is a small, controlled streaming workload: a steady trickle of JSON events being written into a topic over time. Spark's structured streaming job will attach to the same topic and process those events downstream.

Spark Section

Spark Section

We now shift to the Spark layer of the stack.

Kafka handles event transport; Spark handles distributed compute and streaming.

To understand how Spark runs inside our Docker setup, we need to introduce three components:

- Master
- Workers
- Streaming Driver (spark-submit container)

The standalone architecture is simple:

one master coordinates the cluster, workers provide compute resources, and jobs (drivers) request executors from the master to run distributed tasks.

Structured Streaming in One Sentence

Spark's Structured Streaming lets you treat live data as an unbounded table (a continuously growing `DataFrame`). You define transformations once (as if you were writing batch or SQL

code), and Spark incrementally applies them to new data as it arrives, maintaining state and fault tolerance via checkpoints.

Master, Workers, and the Driver

Master

The Spark master is the cluster manager. It:

- Accepts registrations from workers,
- Tracks available cores and memory across the cluster,
- Receives driver submissions,
- Allocates executors to run tasks.

It exposes two important endpoints:

- RPC endpoint used by workers and drivers: `spark://spark-master:7077`
- Web UI at port 8080 (mapped to 8085 on the host)

Workers

Workers are the execution layer. Each worker:

- Connects to the master,
- Offers a configurable number of cores and memory,
- Runs executor JVMs for user jobs,
- Reports status to the master UI.

Workers do not coordinate among themselves; all scheduling decisions come from the master.

Streaming Driver

The streaming driver is the entry point for our Structured Streaming job.

In Spark terminology:

- The **driver program** is the process that runs your application code (our PySpark script).
- A **job** is the unit of work the driver submits to the cluster (e.g. “read this Kafka stream, transform it, write it to S3”).
- **Executors** (on the workers) execute tasks for those jobs in parallel.

In this setup:

- The spark_streaming container executes spark-submit on startup,
- Connects to the master,
- Requests executors,
- Begins consuming from Kafka using Structured Streaming.

The driver holds the streaming query, checkpoint configuration, and control logic; the workers do the heavy lifting on each micro-batch.

Spark Services in Our Docker Compose

The Spark group in docker-compose consists of:

1. spark-master – The standalone master
2. spark_worker_1 – Worker node
3. spark_worker_2 – Worker node
4. spark_streaming – Driver container running our Structured Streaming job

All of these use the same custom Spark image, built from a dedicated Dockerfile.

Why a Custom Spark Image?

Out of the box, many Spark images:

- Do not ship with Python configured the way you want,
- Do not include the Kafka Structured Streaming connector jars,
- Do not contain your application code.

If you try to read from Kafka without the right jars, you get errors such as:

- Failed to find data source: kafka
- ClassNotFoundException: org.apache.spark.sql.kafka010.KafkaSourceProvider

To avoid that, we build a custom Spark image that:

- Starts from a known base (bitnamilegacy/spark:3.5.1),
- Installs python3 so we can run PySpark jobs,
- Copies our streaming application into the image,
- Downloads and pins the exact Kafka + Spark connector jars into Spark's jars directory.

That way, every container (spark-master, workers, and spark_streaming) is consistent and self-contained.

Spark Dockerfile: Line-by-Line Breakdown

FROM bitnamilegacy/spark:3.5.1

- Base image: Bitnami's Spark 3.5.1 distribution.
- Includes Spark binaries and a standard layout under /opt/bitnami/spark.

USER root

- Switch to root so we can install system packages and modify the filesystem.
- Bitnami images typically run as non-root by default; this is an explicit escalation for the build phase only.

RUN set -eux; \

apt-get update; \

apt-get install -y --no-install-recommends \

curl \

python3 \

python3-pip \

python3-venv; \

ln -s /usr/bin/python3 /usr/bin/python || true; \

rm -rf /var/lib/apt/lists/*

This block does all the system-level setup:

- set -eux
 - -e: stop on error
 - -u: error on unset variables
 - -x: print each command (useful for debugging builds).
- apt-get update
 - Refresh package metadata.
- apt-get install ...
 - Install curl (for jar downloads).
 - Install python3, python3-pip, python3-venv so the container has a working Python runtime.

- `ln -s /usr/bin/python3 /usr/bin/python || true`
 - Create a python symlink pointing at python3.
 - Many tools assume python exists; this avoids “command not found”.
- `rm -rf /var/lib/apt/lists/*`
 - Clean up package lists to keep the image smaller.

`RUN mkdir -p /opt/spark/app`

`COPY app/ /opt/spark/app/`

- `mkdir -p /opt/spark/app`
 - Create a directory for your application code.
- `COPY app/ /opt/spark/app/`
 - Copy everything from the local `app/` directory into the image.
 - This is where `spark_processing.py` (your streaming job) lives.
 - Later, `spark-submit` uses `/opt/spark/app/spark_processing.py` as the entry point.

`USER 1001`

- Drop privileges and run as user 1001.
- Matches Bitnami’s security model and is safer than running Spark as root.
- Everything after this runs as a non-root user.

`ARG KAFKA_CLIENT_VERSION=3.5.1`

`ARG SPARK_KAFKA_VERSION=3.5.1`

`ARG SCALA_BINARY_VERSION=2.12`

`ARG COMMONS_POOL_VERSION=2.12.0`

Build-time arguments to control dependency versions:

- `KAFKA_CLIENT_VERSION` – version of kafka-clients.
- `SPARK_KAFKA_VERSION` – version of Spark’s Kafka connector modules.
- `SCALA_BINARY_VERSION` – Scala ABI (must match Spark’s build, here 2.12).
- `COMMONS_POOL_VERSION` – version of Apache Commons Pool used by the connector.

This keeps versioning explicit and easy to change in one place.

```
RUN set -eux; \
```

```
cd /opt/bitnami/spark/jars; \
```

```
curl -fSLo kafka-clients-${KAFKA_CLIENT_VERSION}.jar \
```

```
https://repo1.maven.org/maven2/org/apache/kafka/kafka-clients/${KAFKA_CLIENT_VERSION}/kafka-clients-${KAFKA_CLIENT_VERSION}.jar; \
```

```
curl -fSLo spark-sql-kafka-0-10_${SCALA_BINARY_VERSION}-${SPARK_KAFKA_VERSION}.jar \
```

```
https://repo1.maven.org/maven2/org/apache/spark/spark-sql-kafka-0-10_${SCALA_BINARY_VERSION}/${SPARK_KAFKA_VERSION}/spark-sql-kafka-0-10_${SCALA_BINARY_VERSION}-${SPARK_KAFKA_VERSION}.jar; \
```

```
curl -fSLo spark-token-provider-kafka-0-10_${SCALA_BINARY_VERSION}-${SPARK_KAFKA_VERSION}.jar \
```

```
https://repo1.maven.org/maven2/org/apache/spark/spark-token-provider-kafka-0-10_${SCALA_BINARY_VERSION}/${SPARK_KAFKA_VERSION}/spark-token-provider-kafka-0-10_${SCALA_BINARY_VERSION}-${SPARK_KAFKA_VERSION}.jar; \
```

```
curl -fSLo commons-pool2-${COMMONS_POOL_VERSION}.jar \
```

```
https://repo1.maven.org/maven2/org/apache/commons/commons-pool2/${COMMONS_POOL_VERSION}/commons-pool2-${COMMONS_POOL_VERSION}.jar
```

This is the critical part for Kafka + Spark integration.

- `cd /opt/bitnami/spark/jars`
 - Move into Spark's jar directory.
 - Everything here is automatically added to Spark's classpath.

Each curl download pulls a specific jar from Maven Central:

1. `kafka-clients-${KAFKA_CLIENT_VERSION}.jar`
 - Core Kafka client library.
 - Provides the Kafka consumer API that the Spark Kafka connector wraps.
2. `spark-sql-kafka-0-10_${SCALA_BINARY_VERSION}-${SPARK_KAFKA_VERSION}.jar`
 - Spark SQL Kafka connector for Kafka 0.10+ protocol.
 - Powers `.format("kafka")` in your Spark code.
 - Without this, `spark.readStream.format("kafka")` is not available.

3. spark-token-provider-kafka-0-10-`${SCALA_BINARY_VERSION}`-`${SPARK_KAFKA_VERSION}`.jar
 - Token provider module for secured Kafka (SASL/OAuth, etc.).
 - Even if this demo is unsecured, having it avoids missing-class issues as configs evolve.
4. commons-pool2-`${COMMONS_POOL_VERSION}`.jar
 - Apache Commons Pool library.
 - Used by various connection pooling components in the Kafka connector stack.
 - Missing it can cause `ClassNotFoundException` at runtime.

Because all of these jars end up in `/opt/bitnami/spark/jars`, every Spark process (master, worker, driver) automatically sees them.

That is what makes this one image reusable across all Spark roles in the cluster.

1. spark-master — The Cluster Manager

spark-master:

image: custom-spark

container_name: spark-master

environment:

- SPARK_MODE=master
- SPARK_MASTER_HOST=spark-master
- SPARK_MASTER_PORT=7077
- SPARK_MASTER_WEBUI_PORT=8080
- SPARK_SUBMIT_OPTS=...

ports:

- "8085:8080"

volumes:

- ./:/home
- spark_data:/opt/bitnami/spark/data

networks:

- docker_streaming

- kafka_network

healthcheck: ...

Key points:

- image: custom-spark
 - Uses the custom image built from the Dockerfile above.
- SPARK_MODE=master
 - Tells the entrypoint script to launch the master daemon.
- SPARK_MASTER_HOST=spark-master / SPARK_MASTER_PORT=7077
 - Define the RPC endpoint as spark://spark-master:7077 for workers and drivers.
- SPARK_MASTER_WEBUI_PORT=8080 mapped to host 8085
 - UI at http://localhost:8085.
- Volumes:
 - ./:/home for quick access to host files if needed.
 - spark_data:/opt/bitnami/spark/data for shared internal Spark state and logs.
- Networks:
 - docker_streaming so Airflow and other orchestrators can reach the master.
 - kafka_network for consistent network visibility with the workers and streaming job.
- Healthcheck
 - Ensures the master is fully up before dependent services start, improving cluster stability.

We deploy Spark as a standalone cluster (one master, two workers). This is enough to demonstrate real cluster behaviour—tasks being scheduled across multiple executors—without introducing YARN, Kubernetes, or EMR. In larger deployments you would run the same streaming code against those managers; the core concepts stay the same.

2. spark_worker_1 & spark_worker_2 — Worker Nodes

Each worker uses an almost identical configuration:

spark_worker_1:

image: custom-spark

environment:

- SPARK_MODE=worker
- SPARK_MASTER_URL=spark://spark-master:7077
- SPARK_WORKER_CORES=2
- SPARK_WORKER_MEMORY=4G
- SPARK_WORKER_WEBUI_PORT=8081
- SPARK_SUBMIT_OPTS=...

ports:

- "8086:8081"

networks:

- docker_streaming
- kafka_network

Highlights:

- SPARK_MODE=worker
 - Spawns a Spark worker process.
- SPARK_MASTER_URL=spark://spark-master:7077
 - Tells the worker where to register.
- SPARK_WORKER_CORES=2 / SPARK_WORKER_MEMORY=4G
 - The resource pool this worker offers to executors.
- SPARK_WORKER_WEBUI_PORT=8081 mapped to host 8086 (and 8087 for worker 2)
 - Lets you inspect executor allocations and task metrics.
- Networks: same dual-attachment:
 - kafka_network for consistent addressing with the streaming driver.
 - docker_streaming for visibility from Airflow or other tools.

Two workers are enough to show multi-node computation and see work being distributed across executors in the Spark UI.

3. spark_streaming — The Structured Streaming Driver

This container runs the actual pipeline logic.

On startup it loads AWS/Kafka variables from `.env.aws` and executes:

```
spark-submit \  
--master spark://spark-master:7077 \  
/opt/spark/app/spark_processing.py
```

Configuration summary:

- KAFKA_BOOTSTRAP_SERVERS=kafka:19092
 - Uses Kafka's internal listener on kafka_network.
- KAFKA_TOPIC=names_topic
 - Topic consumed by the streaming job.
- AWS variables from `.env.aws`
 - Credentials for writing results to S3-compatible storage using `s3a://`.
- S3-related environment variables:
 - S3_OUTPUT_PREFIX, S3_CHECKPOINT_PREFIX
 - Define where outputs and checkpoints are written.
- Volumes:
 - ./spark/app:/opt/spark/app
 - Local development code is mounted into the same path the Dockerfile used.
 - spark_data:/opt/bitnami/spark/data
 - Shared internal state volume with the master and workers.
- Networks:
 - kafka_network so the driver can reach Kafka as kafka:19092.
 - docker_streaming so Airflow can monitor and orchestrate it.

The container uses a restart policy; if the Spark job exits unexpectedly, the driver is relaunched. Because Spark uses the checkpoint directory, the streaming job resumes from the last committed offsets.

Networking Strategy: Why Spark Joins Two Networks

The Spark containers attach to:

- `kafka_network`
→ so Spark can consume from Kafka using the internal listener `kafka:19092`.
- `docker_streaming`
→ so Airflow and other non-Kafka services can interact with Spark (UI, job submission, health checks).

This fits the broader networking pattern in the stack:

- Kafka internal communication stays isolated on `kafka_network`.
- Cross-service orchestration flows through `docker_streaming`.

In practice:

- Spark Streaming can reliably reach Kafka.
- Airflow can reliably reach Spark.
- Services do not fight over conflicting advertised listeners or DNS names.

Spark, Hadoop, and `s3a://`

Spark does not talk to S3 directly; it goes through the Hadoop filesystem layer.

- Spark uses the same file APIs that Hadoop uses for HDFS and object stores.
- The `s3a://` scheme is provided by the Hadoop AWS connector (bundled into the image).
- When the job writes to `s3a://my-bucket/prefix/...`, Spark delegates to Hadoop's S3A client, which uses the AWS credentials from the environment.

This is why the Docker image needs both:

- The Spark + Kafka jars (for streaming from Kafka), and
- The Hadoop AWS / S3A stack (for writing to S3-compatible storage).

The result is a Spark job that can stream from Kafka and write directly to an S3 bucket without any additional glue code.

Putting It Together: Kafka → Spark → S3

The pipeline implemented here is a standard streaming pattern:

1. Kafka receives JSON messages from a producer.
2. The spark_streaming driver consumes from names_topic using code of the form:
3. `raw_df = (`
4. `spark.readStream`
5. `.format("kafka")`
6. `.option("kafka.bootstrap.servers", "kafka:19092")`
7. `.option("subscribe", "names_topic")`
8. `.load()`
9. `)`
- 10.
11. `parsed_df = (`
12. `raw_df`
13. `.selectExpr("CAST(value AS STRING) AS json_str")`
14. `.select(`
15. `from_json("json_str", schema).alias("data")`
16. `)`
17. `.select("data.*")`
18. `)`
- 19.
20. `query = (`
21. `parsed_df.writeStream`
22. `.format("json")`
23. `.option("path", f"s3a://{S3_BUCKET}/{S3_OUTPUT_PREFIX}/")`
24. `.option("checkpointLocation", f"s3a://{S3_BUCKET}/{S3_CHECKPOINT_PREFIX}/")`
25. `.outputMode("append")`
26. `.start()`
27. `)`

Step-by-step:

- `spark.readStream.format("kafka")...load()`
Reads the Kafka topic as a streaming DataFrame whose value column is binary.
- `selectExpr("CAST(value AS STRING) AS json_str")`
Casts the Kafka value into a string so it can be parsed as JSON.
- `from_json("json_str", schema)`
Parses each JSON message into structured columns defined by schema.
- Final `select("data.*")`
Flattens the nested struct into normal columns in `parsed_df`.
- `writeStream.format("json")...`
Writes JSON files to `s3a://bucket/prefix` in append mode, using a checkpoint directory for exactly-once semantics and recovery.

28. Workers execute the micro-batches produced by Structured Streaming; the driver coordinates them; the master monitors state across the cluster.

29. The checkpoint at `s3a://{S3_BUCKET}/{S3_CHECKPOINT_PREFIX}/` keeps track of offsets and progress so the query can resume after restarts.

Everything is wired so that once Kafka receives data, Spark continuously ingests it, transforms it using standard DataFrame operations, and sinks it to S3 with fault-tolerant state tracking. The section now shows not just *what* is running (master, workers, driver) but also *how* the Structured Streaming job flows end-to-end through the stack.

Spark Code

Here is a revised Spark Code section with the review points integrated: clearer “big picture” upfront, a short recap of the Structured Streaming model, and tighter stage-by-stage narration (while keeping the technical detail you already have).

Spark Code

Now that Spark is wired into the Docker cluster, we can look at the actual driver script: this module is the **Structured Streaming job** that reads from Kafka and writes to S3-compatible storage.

Before diving into the functions, it is worth stating the execution model explicitly:

Spark Structured Streaming treats each input source (like Kafka) as an **unbounded table**.

Spark continuously appends new data to this table and processes it in **micro-batches**.

Triggers (like `availableNow`) control how long the query runs and when it stops.

At a high level, this job does:

1. Configure a `SparkSession` with S3A (AWS/MinIO) support.
2. Build an **unbounded streaming DataFrame** from Kafka.
3. Parse each Kafka JSON payload into typed columns.
4. Run an S3 health check by writing a small marker file.
5. Stream the transformed data into `s3a://bucket/prefix` using `foreachBatch` and `availableNow`.

The code is organised as **small, focused functions** (Stage 1–5), plus a `main()` orchestrator that wires them together.

Imports and Logging

```
import logging
```

```
import os
```

```
from typing import Optional, Tuple
```

```
from urllib.parse import urlparse
```

```
from datetime import datetime, timezone
```

```
from pyspark.sql import DataFrame, SparkSession
```

```
from pyspark.sql.functions import from_json, col
```

```
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
```

You import:

- Standard library tools: `os`, `datetime`, `logging`, `urlparse`, and typing helpers.
- Spark core types: `SparkSession`, `DataFrame`.
- Spark SQL pieces: `from_json`, `col`, and data types (`StructType`, `StructField`, etc.).

Logging is configured up-front:


```

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s:%(name)s:%(funcName)s:%(levelname)s:%(message)s",
)

logger = logging.getLogger("spark_structured_streaming")

```

This gives structured log lines with timestamp, logger name, function name, level, and message. In Docker/Airflow logs, that makes it substantially easier to trace what happened when.

Stage 1 – Spark + S3 Configuration

Stage 1 is everything concerned with S3A authentication, endpoint handling, and constructing the SparkSession. This is the “plumbing” that lets later stages treat s3a://... paths as normal file system destinations.

_configure_s3_credentials

```

def _configure_s3_credentials(
    builder: SparkSession.Builder,
    access_key: str,
    secret_key: str,
    session_token: Optional[str] = None,
) -> SparkSession.Builder:
    ...

```

Purpose: apply AWS-style credentials directly to the Spark builder in one place.

Key points:

- It chooses the S3A credentials provider based on whether a session token is present:
 - Without a token:


```
org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider
```
 - With a token:


```
org.apache.hadoop.fs.s3a.TemporaryAWSCredentialsProvider
```
- Then it injects:
 - `spark.hadoop.fs.s3a.aws.credentials.provider`

- spark.hadoop.fs.s3a.access.key
- spark.hadoop.fs.s3a.secret.key
- spark.hadoop.fs.s3a.session.token (only when used)

So if you pass `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN`, the job uses short-lived temporary credentials; otherwise it falls back to static keys. All of this is kept in a single helper instead of scattered across the script.

`_normalize_s3_endpoint`

```
def _normalize_s3_endpoint(raw: Optional[str]) -> Tuple[str, Optional[bool]]:
```

```
...
```

Purpose: take a user-supplied `S3_ENDPOINT` and convert it into:

- `host[:port]` (string), and
- `ssl_enabled` (True / False / None).

Supported examples:

- `s3.eu-west-2.amazonaws.com` → ("`s3.eu-west-2.amazonaws.com`", None)
- `http://minio:9000` → ("`minio:9000`", False)
- `https://s3.eu-west-2.amazonaws.com` → ("`s3.eu-west-2.amazonaws.com`", True)

It is deliberately strict:

- No paths, queries, or fragments.
- Only http and https schemes.
- Empty hostnames are rejected.

This prevents mis-specified endpoints like `https://minio:9000/bucket` from “half working” and then failing inside the job. It also makes SSL handling explicit: the function tells you whether SSL should be enabled; you do not have to guess later.

`initialize_spark_session (Stage 1 driver)`

```
def initialize_spark_session(
```

```
    app_name: str,
```

```
    *,
```

```
    region: str,
```

```
    access_key: Optional[str] = None,
```

```
secret_key: Optional[str] = None,  
session_token: Optional[str] = None,  
endpoint: Optional[str] = None,  
path_style: Optional[bool] = None,  
ssl_enabled: Optional[bool] = None,  
) -> SparkSession:
```

...

This is the Stage 1 “entrypoint”: it wires together credentials, region, endpoint, and SSL into a concrete SparkSession.

Configuration flow:

1. **Base builder**

2. builder = SparkSession.builder.appName(app_name)

3. **Credentials**

- If access_key and secret_key are provided → call _configure_s3_credentials(...).
- Otherwise, fall back to:
- "spark.hadoop.fs.s3a.aws.credentials.provider" = \
- "com.amazonaws.auth.DefaultAWSCredentialsProviderChain"

That allows IAM roles or other default providers on “real” AWS infrastructure, and simple env vars in Docker.

4. **Region**

5. builder = builder.config("spark.hadoop.fs.s3a.region", region)

6. **Endpoint**

- If endpoint is provided, set:
- builder = builder.config("spark.hadoop.fs.s3a.endpoint", endpoint)
- For non-amazonaws.com endpoints (for example MinIO), you typically want path-style access, so the code defaults path_style=True unless you override it.

7. **Path-style access**

8. builder = builder.config(

```

9.     "spark.hadoop.fs.s3a.path.style.access",
10.    str(bool(path_style)).lower(),
11. )

```

This is required for MinIO and several other S3-compatible stores. Nuances like bucket policies or VPC-specific endpoints can be considered advanced topics and are not required to understand this script.

12. SSL

If `ssl_enabled` is not `None`, you set:

```

builder = builder.config(
    "spark.hadoop.fs.s3a.connection.ssl.enabled",
    "true" or "false",
)

```

13. Create session

```

14. spark = builder.getOrCreate()

15. spark.sparkContext.setLogLevel("ERROR")

```

On any failure, the exception is logged (with stack trace) and re-raised, so Airflow/Docker sees a clear error rather than a silent misconfiguration.

Stage 2 – Kafka Streaming Source

This is where the Structured Streaming model really starts: we turn a Kafka topic into a streaming `DataFrame`, i.e. an unbounded table that Spark will process in micro-batches.

get_streaming_dataframe

```

def get_streaming_dataframe(spark: SparkSession, brokers: str, topic: str) -> DataFrame:
    """Define a Kafka streaming DataFrame (unbounded table)."""
    return (
        spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", brokers)
        .option("subscribe", topic)
    )

```

```

.option("startingOffsets", "earliest")
.option("failOnDataLoss", "false")
.load()
)

```

Key points:

- `.format("kafka")` uses the Kafka connector jars baked into the image.
- `kafka.bootstrap.servers` is the cluster address (kafka:19092 inside Docker).
- `subscribe` binds this stream to a single topic (here `names_topic`).
- `startingOffsets="earliest"` ensures the job processes the backlog from the beginning.
- `failOnDataLoss="false"` is pragmatic for development/demo; it tolerates some offset gaps when topics are reset.

The returned DataFrame is an **unbounded streaming table** with Kafka metadata columns (key, value, topic, partition, offset, timestamp, etc.), where value is raw bytes. Later stages will stop caring about Kafka internals and just work with structured rows.

Stage 3 – Transforming Kafka JSON Payloads

Stage 3 is where the raw Kafka events become strongly typed columns.

transform_streaming_data

```
def transform_streaming_data(df: DataFrame) -> DataFrame:
```

```
    """Cast Kafka value→STRING, parse JSON to typed columns."""
```

```
    schema = StructType(
```

```
        [
```

```
            StructField("name", StringType(), True),
```

```
            StructField("gender", StringType(), True),
```

```
            StructField("address", StringType(), True),
```

```
            StructField("city", StringType(), True),
```

```
            StructField("nation", StringType(), True),
```

```
            StructField("zip", StringType(), True),
```

```
            StructField("latitude", DoubleType(), True),
```

```

        StructField("longitude", DoubleType(), True),
        StructField("email", StringType(), True),
    ]
)
return (
    df.selectExpr("CAST(value AS STRING) AS json_str")
    .select(from_json(col("json_str"), schema).alias("data"))
    .select("data.*")
)

```

The producer sends JSON-encoded user records as bytes. This function:

1. Defines the expected schema for each event (matching the producer).
2. Performs three transformations:
 - `CAST(value AS STRING)` → convert the Kafka value bytes into a UTF-8 string column `json_str`.
 - `from_json(col("json_str"), schema).alias("data")` → parse JSON into a struct with the given schema.
 - `.select("data.*")` → flatten that struct so the DataFrame has top-level columns: `name`, `gender`, `address`, etc.

Result: a **typed streaming DataFrame** that represents people as rows. At this point, the rest of the pipeline can be reasoned about like any other tabular Spark job.

Stage 4 – S3 Health Check

Stage 4 validates that the configured S3A destination is actually writable **before** you start the streaming sink. This fails fast if credentials, endpoint, or bucket configuration are wrong.

`s3_healthcheck_write`

```
def s3_healthcheck_write(spark: SparkSession, path: str) -> None:
```

```

    """

```

Perform a small healthcheck against S3A by creating a marker file:

```
{path}/healthcheck/_SUCCESS
```

```
.....
```

```
...
```

Steps:

1. Normalise paths:
2. `base = path.rstrip("/")`
3. `health_dir = f"{base}/healthcheck"`
4. `marker = f"{health_dir}/_SUCCESS"`

For `path = "s3a://bucket/names"`, the marker becomes `s3a://bucket/names/healthcheck/_SUCCESS`.

5. Get current timestamp and prepare a payload string.
6. Use the JVM Hadoop API via PySpark's gateway:
7. `jvm = spark._jvm`
8. `hadoop_conf = spark.sparkContext._jsc.hadoopConfiguration()`
9. `Path = jvm.org.apache.hadoop.fs.Path`
- 10.
11. `health_dir_path = Path(health_dir)`
12. `marker_path = Path(marker)`
- 13.
14. `fs = marker_path.getFileSystem(hadoop_conf)`
15. `fs.mkdirs(health_dir_path)`
- 16.
17. `out_stream = fs.create(marker_path, True)`
18. `out_stream.write(payload_bytes)`
19. `out_stream.close()`

`marker_path.getFileSystem(hadoop_conf)` ensures Spark chooses the correct implementation for `s3a://` rather than falling back to the local filesystem.

If anything goes wrong (bad keys, bad endpoint, missing bucket), the exception is logged and re-raised. That is preferable to starting a streaming query that will only fail when it first tries to write a batch.

Stage 5 – Streaming Sink to S3 with availableNow

Stage 5 turns the transformed streaming DataFrame into a **finite** job using Spark's availableNow trigger and foreachBatch.

initiate_streaming_to_bucket

```
def initiate_streaming_to_bucket(df: DataFrame, path: str, checkpoint_location: str) ->
None:
```

```
    """Start JSON streaming sink with checkpointing and per-batch logging (finite run)."""
```

```
    ...
```

Core elements:

1. **Per-batch callback**
2. `def _write_batch(batch_df: DataFrame, batch_id: int) -> None:`
3. `count = batch_df.count()`
4. `ts = datetime.now(timezone.utc).isoformat()`
- 5.
6. `logger.info(`
7. `"Batch %s: about to write %s record(s) to %s at %s",`
8. `batch_id, count, path, ts,`
9. `)`
- 10.
11. `(`
12. `batch_df.write`
13. `.mode("append")`
14. `.format("json")`
15. `.save(path)`
16. `)`


```

17.
18. logger.info(
19.     "Batch %s: successfully wrote %s record(s) to %s at %s",
20.     batch_id, count, path, ts,
21. )

```

For each micro-batch, it:

- Counts the records (for observability).
- Logs before and after writing.
- Writes the batch as JSON with `.mode("append")`.

This gives very clear operational traces: “Batch 0 wrote N records to s3a://bucket/prefix at time T”.

22. Streaming query definition

```

23. query = (
24.     df.writeStream
25.     .outputMode("append")
26.     .foreachBatch(_write_batch)
27.     .option("checkpointLocation", checkpoint_location)
28.     .trigger(availableNow=True) # Spark 3.3+ — drain backlog then stop
29.     .start()
30. )

```

- `outputMode("append")` matches the “new events only” pattern typical for Kafka.
- `foreachBatch` lets you treat each micro-batch like a normal batch DataFrame write.
- `checkpointLocation` stores offsets and progress in S3A.
- `trigger(availableNow=True)` for Spark ≥ 3.3 tells Spark:

Process all data currently available, commit everything, then stop the query.

31. Termination

```

32. query.awaitTermination()

```

```
33. logger.info("Streaming query (availableNow) completed and terminated.")
```

So Stage 5 is a **bounded** Structured Streaming job: it drains the current Kafka backlog once, writes everything, and then exits. That behaviour is much easier to integrate with Airflow than an infinite streaming service.

Orchestrator – main()

main() is the end-to-end orchestrator: Stage 1 → 4 → 2 → 3 → 5, in a clean, linear sequence.

def main() -> None:

```
"""End-to-end: Spark init → Kafka read → JSON parse → JSON sink (AWS S3)."""
```

```
app_name = "spark_streaming"
```

```
brokers = os.environ.get("KAFKA_BOOTSTRAP_SERVERS", "kafka:19092")
```

```
topic = os.environ.get("KAFKA_TOPIC", "names_topic")
```

```
bucket = os.environ.get("S3_BUCKET")
```

```
if not bucket:
```

```
    raise RuntimeError("S3_BUCKET is required")
```

```
output_prefix = os.environ.get("S3_OUTPUT_PREFIX", "names").strip("/")
```

```
checkpoint_prefix = (
```

```
    os.environ.get("S3_CHECKPOINT_PREFIX") or f"checkpoints/{output_prefix}"
```

```
).strip("/")
```

```
path = f"s3a://{bucket}/{output_prefix}"
```

```
checkpoint_location = f"s3a://{bucket}/{checkpoint_prefix}"
```

Block 1 – destinations

- KAFKA_BOOTSTRAP_SERVERS / KAFKA_TOPIC select the Kafka source.
- S3_BUCKET is mandatory.

- S3_OUTPUT_PREFIX and S3_CHECKPOINT_PREFIX control where JSON outputs and checkpoints are written (with sensible defaults).
- Full S3A URIs are assembled for both path and checkpoint_location.

```
s3_region = os.environ.get("S3_REGION") or os.environ.get("AWS_REGION") or "eu-west-2"
```

```
access_key = os.environ.get("AWS_ACCESS_KEY_ID")
```

```
secret_key = os.environ.get("AWS_SECRET_ACCESS_KEY")
```

```
session_token = os.environ.get("AWS_SESSION_TOKEN")
```

Block 2 – S3 auth

Pull region and credentials from the environment. In local Docker usage these are usually static env vars; on real AWS this could be IAM roles.

```
endpoint_host = None
```

```
ssl_pref = None
```

```
endpoint_raw = os.environ.get("S3_ENDPOINT")
```

```
if endpoint_raw:
```

```
    endpoint_host, ssl_pref = _normalize_s3_endpoint(endpoint_raw)
```

```
path_style_env = os.environ.get("S3_PATH_STYLE_ACCESS")
```

```
path_style = None
```

```
if path_style_env is not None:
```

```
    path_style = path_style_env.lower() in {"1", "true", "yes", "on"}
```

Block 3 – endpoint and path-style

- S3_ENDPOINT is normalised; if present, you get: endpoint_host (e.g. minio:9000) and ssl_pref (True/False).
- S3_PATH_STYLE_ACCESS can explicitly override path-style behaviour.

Then the entire configuration is logged in one line:

```
logger.info(
```

```
    "Spark streaming config: app_name=%s brokers=%s topic=%s bucket=%s "
```

```

"output_prefix=%s checkpoint_prefix=%s region=%s endpoint=%s "
"path_style=%s ssl_enabled=%s",
app_name,
brokers,
topic,
bucket,
output_prefix,
checkpoint_prefix,
s3_region,
endpoint_host,
path_style,
ssl_pref,
)

```

This makes debugging misconfigurations straightforward.

Next, Stage 1 is invoked:

```

spark = initialize_spark_session(
    app_name,
    region=s3_region,
    access_key=access_key,
    secret_key=secret_key,
    session_token=session_token,
    endpoint=endpoint_host,
    path_style=path_style,
    ssl_enabled=ssl_pref,
)

```

Then the Stage 4 → 2 → 3 → 5 pipeline, in order:

try:

```

# Stage 4 – explicit S3 connectivity check

```

```
s3_healthcheck_write(spark, path)
```

```
# Stage 2 – Kafka streaming source
```

```
df = get_streaming_dataframe(spark, brokers, topic)
```

```
# Stage 3 – JSON parsing / struct→columns
```

```
transformed_df = transform_streaming_data(df)
```

```
# Stage 5 – drain backlog to S3 with checkpointing
```

```
initiate_streaming_to_bucket(transformed_df, path, checkpoint_location)
```

```
finally:
```

```
    logger.info("Stopping Spark session.")
```

```
    spark.stop()
```

```
Entry point:
```

```
if __name__ == "__main__":
```

```
    main()
```

```
This allows:
```

- spark-submit to treat this file as the main driver, and
- Unit tests to import functions and main() with mocked env vars and test endpoints.

Summary: How This Code Fits into the Stack

Putting the Spark layer back into the broader pipeline:

- The **Kafka producer** continually writes clean JSON records into names_topic.
- The **Spark driver** (this script) runs inside the spark_streaming container, connects to spark-master, and uses the workers to run micro-batches.
- Spark **treats the Kafka topic as an unbounded table**, parses each JSON record into typed columns, and, via availableNow, performs a **one-shot drain** of the current backlog.

- The result is written as JSON into an S3-compatible bucket with explicit S3A configuration, a pre-flight health check, checkpointing, and per-batch logging.

Kafka provides the stream of events; this Spark code is the stage-by-stage bridge that turns those events into durable, queryable objects in S3, with enough structure and logging to be operable in a real data engineering workflow.

Airflow Section

Airflow Section

We now move to the orchestration layer: Apache Airflow.

Kafka moves messages, Spark does the heavy computation, and Airflow is the “brain” that decides **when** each piece runs and **in what order**.

At a high level, Airflow in this stack is made of four core pieces:

- **Scheduler** – reads DAGs, decides which tasks should run, and when.
- **Executor** – actually runs those tasks (LocalExecutor in this setup).
- **Webserver** – UI + API to inspect and control workflows.
- **Metadata Database** – PostgreSQL backing store for all state.

All of this runs on a single machine via Docker Compose, using **LocalExecutor**.

Core Concepts: Scheduler, Webserver, Metadata DB, Executor

Scheduler

The scheduler is the central loop. It:

- Continuously scans the dags/ folder,
- Parses DAG files into an internal graph,
- Decides when each task becomes runnable (based on schedule, dependencies, and state),
- Hands runnable tasks off to the executor.

In **LocalExecutor** mode, the scheduler process itself spawns subprocesses to run tasks in parallel, instead of handing them to a remote worker cluster.

Webserver (UI/API)

The webserver:

- Hosts the Airflow web UI and REST API,
- Reads all state from the metadata database,
- Shows DAGs, task statuses, logs, and configuration,
- Lets you trigger DAGs, pause/unpause, clear tasks, etc.

It does **not** execute tasks. It only reads/writes metadata and exposes it via HTTP.

Metadata Database (PostgreSQL)

Airflow is state-heavy. The metadata DB holds:

- DAG definitions (parsed versions),
- Task instances and their statuses,
- XComs (small cross-task messages),
- Variables, connections, users, roles, and more.

In this stack, the metadata DB is a dedicated PostgreSQL container (airflow_db).

Every Airflow component (scheduler, webserver, init job) connects to it over the internal Docker network.

Executor Modes (focus: LocalExecutor)

Airflow supports several executor backends:

- SequentialExecutor – single process, one task at a time; only for development.
- LocalExecutor – multiple tasks in parallel as subprocesses on **one** machine.
- CeleryExecutor, KubernetesExecutor, etc. – distribute work across many workers/nodes.

This project uses **LocalExecutor**, which provides:

- Real parallelism on a single host (multiple Python processes),
- No external queue or worker farm,
- A simpler setup for a local Docker stack.

Scheduler configuration (parallelism, max_active_tasks_per_dag, etc.) controls how many tasks can run concurrently.

How Airflow Orchestrates “Streaming” Here

Airflow is fundamentally a **batch scheduler**: it runs DAGs at discrete times (cron-style), not as infinite daemons.

Spark Structured Streaming, on the other hand, is usually long-running.

To reconcile these two:

- In this demo, each Airflow run triggers a **short-lived Spark job** (via DockerOperator).
- That Spark job processes new data since the last run (or a small time window) and then exits.
- You can schedule the DAG very frequently (e.g. every minute) so that, from a distance, it behaves like a near-real-time pipeline.

So conceptually:

Kafka is continuously receiving events.

Airflow wakes up on a schedule, checks system health, then starts a finite Spark job that consumes “the latest slice” of data and lands it into S3.

This is how we simulate streaming behaviour while still using Airflow’s batch-oriented execution model.

Airflow Services in Docker Compose

In docker-compose.yml, the Airflow setup is split into four services:

1. airflow_db – PostgreSQL metadata database
2. airflow_init – one-shot initialization job
3. airflow_webserver – UI/API
4. airflow_scheduler – scheduler and LocalExecutor

All of them:

- Use the same **custom Airflow image**,
- Share the same DAGs/logs/plugins volumes,
- Run on the same docker_streaming network alongside Kafka and Spark.

Before looking at each service, we need the image they share.

Airflow Dockerfile: Why We Build a Custom Image

All Airflow services use the same custom image, built from:


```
FROM apache/airflow:2.9.3-python3.11
```

```
ARG AIRFLOW_VERSION=2.9.3
```

```
ARG PYTHON_MAJOR_MINOR=3.11
```

```
ARG
```

```
AIRFLOW_CONSTRAINTS_URL="https://raw.githubusercontent.com/apache/airflow/constraints-${AIRFLOW_VERSION}/constraints-${PYTHON_MAJOR_MINOR}.txt"
```

```
USER airflow
```

```
COPY --chown=airflow:airflow requirements.txt /tmp/requirements.txt
```

```
RUN pip install --no-cache-dir \  
    --constraint "${AIRFLOW_CONSTRAINTS_URL}" \  
    -r /tmp/requirements.txt \  
    && rm -f /tmp/requirements.txt
```

Purpose of the Custom Image

The base apache/airflow image does not include:

- confluent-kafka (for Kafka health checks, producers, consumers),
- pyspark (if tasks interact with Spark or inspect Spark output),
- requests, json5, or any other library you call from PythonOperator tasks.

This Dockerfile fixes that by:

1. Starting from the official apache/airflow image.
2. Respecting Airflow's **constraints file** to keep dependencies compatible.
3. Installing your project-specific Python requirements.
4. Ensuring **all** Airflow components (scheduler, webserver, init job) share the exact same Python environment.

Why the Constraints File Matters

Airflow pins its dependency versions. If you install arbitrary package versions, you risk breaking the scheduler or webserver in subtle ways.

The constraints file tells pip which specific versions are safe. You get:

- Extra libraries you need,
- Without fighting Airflow's dependency graph.

Why COPY requirements.txt

Copying requirements.txt into the image means:

- Kafka clients can be imported directly inside DAG tasks.
- Airflow can trigger and control Spark-related scripts without extra hacks.
- PythonOperator tasks can use any installed library without per-task container gymnastics.

Without this custom image, tasks would fail with errors such as `ModuleNotFoundError`.

1. airflow_db – Metadata Database

This is a plain PostgreSQL container dedicated to Airflow:

airflow_db:

image: postgres:16.0

environment:

- POSTGRES_USER=\${POSTGRES_USER} # e.g. airflow
- POSTGRES_PASSWORD=\${POSTGRES_PASSWORD} # e.g. airflow
- POSTGRES_DB=\${POSTGRES_DB} # e.g. airflow_db

volumes:

- ./airflow_pgdata:/var/lib/postgresql/data

logging:

options:

max-size: 10m

max-file: "3"

healthcheck:

test: ["CMD-SHELL", "pg_isready -U \${POSTGRES_USER} -d \${POSTGRES_DB}"]

interval: 10s

timeout: 5s

start_period: 90s

retries: 10

networks:

- docker_streaming

Key points:

- **Environment variables:**
POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB define DB credentials and the database name.
Airflow uses these in its SQL_ALCHEMY_CONN string.
- **Volume:**
./airflow_pgdata:/var/lib/postgresql/data persists the database files, so Airflow state survives container restarts.
- **Healthcheck:**
Uses pg_isready to verify PostgreSQL is actually accepting connections.
Other services can use depends_on: condition: service_healthy to wait for it.
- **Network:**
Runs on docker_streaming, so every Airflow component, plus Kafka and Spark, can reach it via the service name airflow_db.

2. airflow_init – One-Time Initialization

airflow_init is a short-lived container that:

- Runs Airflow database migrations,
- Creates the initial admin user,
- Then exits.

Typical pattern:

airflow_init:

<<: *airflow-common # shared image/env (executor, DB URL, etc.)

depends_on:

airflow_db:

condition: service_healthy

command: >

```
bash -c "  
    airflow db migrate &&  
    airflow users create  
        --username ${AIRFLOW_ADMIN_USERNAME}  
        --password ${AIRFLOW_ADMIN_PASSWORD}  
        --firstname Admin  
        --lastname User  
        --role Admin  
        --email ${AIRFLOW_ADMIN_EMAIL}  
"
```

networks:

- docker_streaming

Key points:

- Uses the same image and environment as the webserver/scheduler (shared *airflow-common anchor).
- Runs exactly once; after successful completion it should not restart.
- Depends on airflow_db health so migrations do not run against an unready DB.
- Shares the same DAGs/logs/plugins volumes as the main Airflow services.

By isolating initialization:

- Migrations do not race the scheduler or webserver.
- Failures are visible in a single, contained phase.
- Main services can assume a ready, migrated metadata DB.

3. airflow_webserver – Airflow UI/API

The webserver is the HTTP entry point:

airflow_webserver:

<<: *airflow-common

depends_on:

airflow_db:

condition: service_healthy

airflow_init:

condition: service_completed_successfully

command: airflow webserver

ports:

- "8080:8080"

healthcheck:

test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]

interval: 10s

timeout: 10s

retries: 5

networks:

- docker_streaming

Key points:

- **Command:** airflow webserver starts the Flask-based UI + REST API.
- **Ports:** container port 8080 is mapped to host 8080 for browser access.
- **Environment (via *airflow-common)** typically includes:
 - AIRFLOW__CORE__EXECUTOR=LocalExecutor
 - AIRFLOW__CORE__SQL_ALCHEMY_CONN=postgresql+psycopg2://\${POSTGRES_USER}:\${POSTGRES_PASSWORD}@airflow_db/\${POSTGRES_DB}
 - Optional auth settings, DAG pause behaviour, etc.
- **Volumes** (also defined in *airflow-common):
 - ./dags:/opt/airflow/dags
 - ./logs:/opt/airflow/logs
 - ./plugins:/opt/airflow/plugins

Dropping a file into ./dags on the host makes it visible to both webserver and scheduler.

- **Healthcheck:**
Polls /health to confirm the UI is responsive. Docker marks the container unhealthy if this fails repeatedly.
 - **Startup order:**
Waits for airflow_db to be healthy and airflow_init to complete successfully.
-

4. airflow_scheduler – Scheduler + LocalExecutor

This container runs the scheduling loop and spawns task processes:

airflow_scheduler:

<<: *airflow-common

depends_on:

airflow_db:

condition: service_healthy

airflow_init:

condition: service_completed_successfully

command: airflow scheduler

networks:

- docker_streaming

Key points:

- Same image and environment as the webserver: same executor, same DB URL, same DAGs/logs/plugins volumes.
- **Command:** airflow scheduler starts the scheduler daemon.
- **Executor:** LocalExecutor runs tasks as subprocesses inside this container.
- No exposed ports: it does not serve HTTP; it talks to the metadata DB and external systems (Kafka, Spark, S3, etc.).
- Usually configured with restart: always so it comes back if it crashes.

The scheduler + LocalExecutor is what actually runs your Kafka producers, Kafka/S3 health checks, Spark triggers, and any other operators you define.

Health-Check Tasks and DAG Structure

The main DAG for this guide is a **streaming pipeline DAG** (for example, `name_stream_dag`) that coordinates Kafka, Spark, and S3.

Rather than directly starting Spark, it uses a series of **health-check tasks** to make sure the rest of the stack is ready.

Typical health-check tasks:

- `kafka_health_check`
 - Uses Kafka client code (e.g. `confluent-kafka`) from a `PythonOperator`.
 - Confirms the broker at `kafka:9092` is reachable.
 - Optionally checks that the target topic exists.
- `s3_bucket_health_check`
 - Uses AWS SDK / `boto3` or a CLI command via `DockerOperator`.
 - Verifies that the target S3 bucket is reachable and listable.
 - Fails early if credentials or permissions are wrong.

You can extend this pattern with a Spark health check if needed (e.g. probing `spark-master:7077` or the Spark web UI).

These tasks act as **system checkpoints**: if any fails, the DAG stops before launching Spark. That prevents you from starting a streaming job into a broken environment.

At a high level, the DAG structure looks like:

`kafka_health_check`

↓

`s3_bucket_health_check`

↓

`spark_stream_to_s3`

↓

`verify_s3_results` / downstream checks

- The **health-check tasks** ensure Kafka and S3 are ready.
- `spark_stream_to_s3` (typically a `DockerOperator`) starts a short-lived Spark streaming job in a dedicated container.
- A final verification task can check that JSON files appeared in the expected S3 path, or even run an Athena query as a smoke test.

This gives a clear mental model of what Airflow is doing each run, before you dig into the full DAG code.

Integration with Kafka and Spark

This Airflow instance is not isolated; it orchestrates the rest of the stack.

Kafka

Because Airflow services live on the `docker_streaming` network, they can:

- Reach the Kafka broker via `kafka:9092` (or `kafka:19092` depending on listeners),
- Run tasks that:
 - Produce test messages into Kafka,
 - Check topic existence and broker health,
 - Monitor readiness conditions before starting Spark.

The custom Airflow image includes Kafka client libraries (e.g. `confluent-kafka`), so `PythonOperator` tasks can talk to Kafka directly from inside the Airflow containers.

Spark

Airflow also controls Spark:

- Triggers batch or short-lived streaming jobs,
- Uses `DockerOperator` to start a driver container (for example, `spark_stream_to_s3`),
- Or uses Spark operators / shell commands if Spark is exposed at `spark://spark-master:7077`.

Because everything shares the `docker_streaming` network:

- Airflow resolves `spark-master` by name and can call its RPC endpoint.
- It can coordinate sequencing such as:

“Check Kafka and S3 → start Spark streaming container → optionally verify outputs → trigger downstream tasks.”

The overall pattern:

- Kafka handles real-time message transport.
- Spark performs streaming and batch computation.

- Airflow orchestrates **when** components run, handles retries, dependencies, and scheduling across the entire pipeline.
-

Network: `docker_streaming`

All Airflow, Kafka, and Spark services join the same user-defined network:

networks:

`docker_streaming`:

driver: bridge

This gives you:

- Stable service-name DNS (`airflow_webserver`, `airflow_scheduler`, `airflow_db`, `kafka`, `spark-master`, etc.),
 - Isolation from unrelated containers on the host,
 - A single logical “data platform” network where orchestration, messaging, and compute can talk to each other without extra host-level port wrangling.
-

This Airflow setup gives you a fully containerized orchestrator:

- One PostgreSQL service for state (`airflow_db`),
- One init job to prepare the environment (`airflow_init`),
- One UI/API for visibility (`airflow_webserver`),
- One scheduler running LocalExecutor (`airflow_scheduler`),

all wired into Kafka and Spark via `docker_streaming`, with health-check tasks and short-lived Spark jobs providing a controlled, Airflow-driven approximation of streaming in your local data platform.

The DAG

The DAG

With Kafka, Spark, and Airflow wired up, the last piece is the DAG that drives the streaming pipeline.

This Python module (dags/kafka_stream_dag.py) is the DAG definition. Airflow discovers it by scanning the dags/ folder, imports the file, and the with DAG(...) as dag: block below is what actually registers the tasks you see in the UI.

The DAG is intentionally small. The business logic (producing names, reading from Kafka, writing to S3) lives in:

- the Kafka producer module (producer.kafka_streaming_service), and
- the Spark job (spark_processing.py).

The DAG's job is to orchestrate and guard the pipeline using four control tasks:

1. Check Kafka and the target topic.
2. Check the S3 bucket (and create it if necessary).
3. Run the Kafka producer.
4. Launch the Spark streaming consumer as a separate container.

So the structure is:

Kafka health check → S3 health check → Producer → Spark consumer

That's the entire control plane.

File: dags/kafka_stream_dag.py

Purpose: Run a short-lived Kafka streaming job on a schedule via Airflow.

Location: Place this file in your Airflow `dags/` directory.

from datetime import timedelta

import pendulum

from airflow import DAG

from airflow.operators.python import PythonOperator # for Option A (spark-submit inside Airflow)

```
from airflow.providers.docker.operators.docker import DockerOperator # for Option B
(Spark container)
```

```
# Reuse the same Kafka configuration (bootstrap servers, topic name)
```

```
# that we defined in the producer service in Stage 1.
```

```
from producer.kafka_streaming_service import ensure_topic, KAFKA_BOOTSTRAP,
KAFKA_TOPIC
```

```
import os
```

```
import boto3
```

```
from botocore.exceptions import ClientError
```

```
from airflow.exceptions import AirflowException
```

```
# -----
```

```
# Callables used by tasks
```

```
# -----
```

```
def _trigger_stream() -> None:
```

```
    """Import and run the Kafka streaming helper at task runtime.
```

```
    Importing inside the callable keeps the DAG importable even if optional
    runtime dependencies (e.g., ``confluent_kafka``) are missing when the
    scheduler parses the file.
```

```
    """
```

```
    from producer.kafka_streaming_service import initiate_stream
```

```
    initiate_stream()
```

```
def _check_kafka() -> None:
```

```
    """Ensure Kafka broker is reachable and the topic exists.
```

Uses the same KAFKA_BOOTSTRAP/KAFKA_TOPIC constants as the producer
so configuration stays consistent end-to-end.

```
"""
```

```
# This will no-op if the topic already exists
```

```
ensure_topic(KAFKA_BOOTSTRAP, KAFKA_TOPIC, num_partitions=1, replication_factor=1)
```

```
def _check_bucket() -> None:
```

```
    """Ensure AWS S3 bucket exists; create it if missing.
```

```
    - 404 / NoSuchBucket: create the bucket in our region.
```

```
    - 403 / AccessDenied: fail fast with clear message (name taken or no perms).
```

```
    """
```

```
    bucket = "namegeneratorbucket"
```

```
    region = os.getenv("AWS_REGION", "eu-west-2")
```

```
    s3 = boto3.client("s3", region_name=region)
```

```
    try:
```

```
        s3.head_bucket(Bucket=bucket)
```

```
        # If we get here, bucket is accessible and exists; health check passes.
```

```
        return
```

```
    except ClientError as e:
```

```
        code = e.response["Error"]["Code"]
```

```
        # Bucket not found → we try to create it in our account.
```

```
        if code in ("404", "NoSuchBucket", "NotFound"):
```

```
            if region == "us-east-1":
```

```

        s3.create_bucket(Bucket=bucket)
    else:
        s3.create_bucket(
            Bucket=bucket,
            CreateBucketConfiguration={"LocationConstraint": region},
        )
    return

# Access denied / forbidden: likely bucket name already taken or IAM blocked
if code in ("403", "AccessDenied"):
    raise AirflowException(
        f"S3 bucket health check failed: cannot access bucket '{bucket}' "
        f"(HTTP {code}). The name may be taken by another account or your "
        f"IAM principal lacks s3:HeadBucket/ListBucket permissions."
    ) from e

# Any other error: re-raise
raise

# -----
# DAG default args and config
# -----

LOCAL_TZ = pendulum.timezone("Europe/London")
DAG_START_DATE = pendulum.datetime(2024, 1, 1, 0, 0, tz=LOCAL_TZ)

DEFAULT_ARGS = {
    "owner": "airflow",

```

```
"start_date": DAG_START_DATE,  
"retries": 1,  
"retry_delay": timedelta(seconds=5),  
}
```

```
# -----
```

```
# DAG DEFINITION
```

```
# -----
```

```
with DAG(  
    dag_id="name_stream_dag",  
    description="Stream random names to Kafka and land them in S3 via Spark",  
    default_args=DEFAULT_ARGS,  
    schedule="*/5 * * * *",  
    catchup=False,  
    max_active_runs=1,  
    tags=["kafka", "streaming", "demo"],  
) as dag:
```

```
# 1) Kafka health check
```

```
kafka_health_check = PythonOperator(  
    task_id="kafka_health_check",  
    python_callable=_check_kafka,  
)
```

```
# 2) S3 / MinIO bucket health check
```

```
bucket_health_check = PythonOperator(  
    task_id="s3_bucket_health_check",
```

```
python_callable=_check_bucket,  
)
```

3) Kafka producer (existing logic)

```
kafka_stream_task = PythonOperator(  
    task_id="stream_to_kafka_task",  
    python_callable=_trigger_stream,  
)
```

4) Spark consumer: choose ONE of the following implementations

```
# -----
```

DockerOperator (Spark runs in its own container)

```
# -----
```

```
spark_stream_task = DockerOperator(  
    task_id="spark_stream_to_s3",  
    image="custom-spark",  
    command=(  
        "/opt/bitnami/spark/bin/spark-submit "  
        "--master spark://spark-master:7077 "  
        "/opt/spark/app/spark_processing.py"  
    ),  
    network_mode="docker_streaming",  
    mount_tmp_dir=False, # <-- important for docker-in-docker / remote engine  
    environment={  
        # Kafka  
        "KAFKA_BOOTSTRAP_SERVERS": "kafka:19092",  
        "KAFKA_TOPIC": "names_topic",
```

```

# S3 / AWS – forward from Airflow’s own environment

"AWS_ACCESS_KEY_ID": os.environ.get("AWS_ACCESS_KEY_ID", ""),
"AWS_SECRET_ACCESS_KEY": os.environ.get("AWS_SECRET_ACCESS_KEY", ""),
"AWS_SESSION_TOKEN": os.environ.get("AWS_SESSION_TOKEN", ""),
"AWS_REGION": os.environ.get("AWS_REGION", "eu-west-2"),

# App-specific config

"S3_BUCKET": "namegeneratorbucket",
"S3_OUTPUT_PREFIX": "names",
"S3_CHECKPOINT_PREFIX": "checkpoints/names",
"S3_REGION": "eu-west-2",
},
auto_remove=True,
execution_timeout=timedelta(minutes=5),
)

# 5) Wiring: health checks → producer → Spark consumer

kafka_health_check >> bucket_health_check >> kafka_stream_task >> spark_stream_task

```

Four control tasks, not one monolith

The pipeline work is mostly automated once services are up. This DAG acts as a set of guardrails.

1. kafka_health_check – verify messaging before anything else

- **Task:** kafka_health_check
- **Callable:** _check_kafka

What it does:

- Calls ensure_topic(KAFKA_BOOTSTRAP, KAFKA_TOPIC, ...).

- Validates that the Kafka broker is reachable at the configured bootstrap address.
- Ensures the topic exists (creates it if missing, no-ops if present).
- Uses the same KAFKA_BOOTSTRAP / KAFKA_TOPIC constants that were introduced in the Kafka producer section, so the DAG and producer share a single source of truth for Kafka config.

Why it matters:

- If Kafka is down or misconfigured, the DAG fails here with a clear error instead of letting later tasks fail in more opaque ways.
 - This is the first guardrail: no healthy topic, no pipeline.
-

2. s3_bucket_health_check – assert storage is available

- **Task:** s3_bucket_health_check
- **Callable:** _check_bucket

What it does:

- Uses boto3 to call head_bucket on namegeneratorbucket in AWS_REGION.
- If the bucket does not exist in your account (404 / NoSuchBucket / NotFound), it creates it in the correct region.
- If the bucket exists but is not accessible (403 / AccessDenied), it raises an AirflowException with an explicit explanation.
- Any other S3 error is re-raised.

Why it matters:

- Guarantees that the downstream Spark job has a valid and accessible S3/MinIO target.
 - Automatically bootstraps the bucket for a “first run”, but fails fast if the bucket name is actually owned elsewhere or IAM is wrong.
 - This is the second guardrail: no writable bucket, no consumer.
-

3. stream_to_kafka_task – run the producer in a controlled way

- **Task:** stream_to_kafka_task
- **Callable:** _trigger_stream

What it does:

- Imports `initiate_stream` at task runtime instead of at DAG parse time.
- Calls `initiate_stream()` from `producer.kafka_streaming_service` — the same module you used earlier to build and test the producer.

Why the late import is deliberate:

- The DAG stays importable even if `confluent_kafka` or other optional runtime dependencies are not present on the scheduler.
- Failures due to missing Kafka libraries appear as task failures in the UI, rather than crashing the scheduler during DAG parse.

Conceptually, this task is the **producer side** of the pipeline: it sends a burst of random name events into the target Kafka topic, but only after Kafka and S3 have passed their health checks.

4. `spark_stream_to_s3` – delegate consumption to a Spark driver container

- **Task:** `spark_stream_to_s3`
- **Operator:** `DockerOperator`

What it does:

- Spawns a disposable container using the custom-spark image.
- Runs `spark-submit` with:
 - `--master spark://spark-master:7077` to target your standalone Spark master.
 - `/opt/spark/app/spark_processing.py` as the driver script.
- Joins the `docker_streaming` network so it can resolve:
 - `kafka` (for Kafka I/O),
 - `spark-master` (for executor allocation).
- Injects configuration via environment:
 - Kafka (`KAFKA_BOOTSTRAP_SERVERS`, `KAFKA_TOPIC`),
 - AWS credentials (forwarded from Airflow's own environment),
 - S3 details (bucket, prefixes, region).

Operational aspects:

- `mount_tmp_dir=False` avoids Airflow's default temp volume behaviour, which can complicate Docker-in-Docker and remote execution setups.
- `auto_remove=True` ensures the driver container is cleaned up automatically.
- `execution_timeout=5 minutes` prevents a stuck job from running indefinitely.

This is the **consumer side** of the pipeline: Spark reads from the Kafka topic and writes the results into S3, relying on the earlier control tasks to guarantee prerequisites.

Wiring: enforcing order in an automated pipeline

The dependency line:

```
kafka_health_check >> bucket_health_check >> kafka_stream_task >> spark_stream_task
```

turns the four tasks into a strict sequence:

1. Validate Kafka.
2. Validate / prepare S3.
3. Run the producer.
4. Run the Spark consumer.

Most of the work is done by services and separate modules:

- Kafka producer module → pushes events.
- Kafka broker/topic → transports events.
- Spark job → consumes from Kafka, writes to S3.
- S3 bucket → stores the JSON output.

This DAG file, living in `dags/kafka_stream_dag.py`, is the control plane that ties all of that together. It does not embed the business logic into Airflow; it enforces a clean, linear contract:

- Do not produce if Kafka is broken.
- Do not consume if storage is unavailable.
- Do not launch Spark if upstream checks have not completed cleanly.

That is how this single DAG definition connects back to the earlier Kafka and Spark code and turns the whole stack into a scheduled, end-to-end streaming pipeline.

End Results

Here's a clean ending section you can drop in.

End Results: What You Should See in S3

To avoid turning this into an AWS account/IAM walkthrough, I will skip over how to set up AWS itself, IAM Identity Center, and so on. The assumption is:

- You already have AWS credentials configured for the Airflow stack.
- `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION` are set.
- The DAG has been run at least once successfully.

Given that, the end result of running `name_stream_dag` is:

- **JSON files** written into your S3 bucket `namegeneratorbucket`
- Under the prefix:
`names/` (as configured via `S3_OUTPUT_PREFIX`)

A typical layout in the bucket will look roughly like:

- `s3://namegeneratorbucket/names/...` – data files written by the Spark job
- `s3://namegeneratorbucket/checkpoints/names/...` – streaming checkpoints

The exact filenames will depend on the Spark configuration (partitioning, number of files, etc.), but you should see a set of `.json` files arriving under `names/` every time the DAG runs and the Spark step succeeds.

Each JSON record corresponds to one message that moved all the way through the pipeline:

1. Produced into Kafka by `stream_to_kafka_task`.
2. Consumed by Spark from the Kafka topic.
3. Written out to S3 as JSON.

At this point, the streaming loop is working end-to-end. The next step is to **query** those files without building a full data warehouse. That is where Athena comes in.

Querying the Output with Athena

What Athena is (and why we use it here)

Amazon Athena is a serverless, interactive query service that lets you run SQL directly against data stored in S3. You do not provision or manage any servers; you just:

- Tell Athena **where** your data lives in S3.
- Describe the **schema**.
- Run standard SQL queries.

In this pipeline, Athena's role is simple:

- Provide a low-friction way to **verify** that the JSON files written by Spark are correct and readable.
- Let you run quick exploratory queries (counts, filters, sampling) over the streaming output without moving it into another system.

High-level Athena setup (only what we need)

Assuming you are already in the Athena console and have configured a query result location in S3, the minimal steps are:

1. **Create a database** for this demo (once):
2. `CREATE DATABASE IF NOT EXISTS demo_streaming;`
3. **Create an external table** over the JSON files in `namegeneratorbucket/names/`.

The exact columns depend on what `spark_processing.py` writes out.

Suppose the JSON structure looks like:

```
{
  "name": "Alice",
  "source": "kafka-name-generator",
  "event_timestamp": "2024-01-01T12:34:56Z"
}
```

Then you can define:

```
CREATE EXTERNAL TABLE IF NOT EXISTS demo_streaming.names_stream (
  name      string,
  source     string,
```

```
event_timestamp timestamp
)
ROW FORMAT SERDE 'org.openx.data.jsonserde.JsonSerDe'
LOCATION 's3://namegeneratorbucket/names/';
```

Adjust the column list to match your actual JSON schema.

Example queries to validate the pipeline

Once the table exists, you can run simple queries to validate the end-to-end path.

1. **Check that data arrived at all:**
2. SELECT COUNT(*) AS row_count
3. FROM demo_streaming.names_stream;

If this returns a non-zero count and keeps increasing over time, your DAG is successfully:

- Producing to Kafka,
- Consuming with Spark,
- Writing to S3.

4. **Inspect a small sample of events:**
5. SELECT
6. name,
7. source,
8. event_timestamp
9. FROM demo_streaming.names_stream
10. ORDER BY event_timestamp DESC
11. LIMIT 10;

This should show the most recent names written by the streaming job, with their timestamps.

12. **Sanity-check by time window (optional):**
13. SELECT
14. date_trunc('hour', event_timestamp) AS hour_bucket,
15. COUNT(*) AS events_per_hour

16. FROM demo_streaming.names_stream

17. GROUP BY 1

18. ORDER BY 1 DESC;

This lets you confirm that events are arriving at roughly the cadence you expect from the scheduled DAG runs.

At this point you have:

- Airflow orchestrating the pipeline and acting as a control layer.
- Kafka and Spark handling streaming ingestion and processing.
- S3 storing the results as JSON.
- Athena providing a simple SQL surface to inspect and validate the output.

That combination is enough to go from “local Docker stack” to something that looks and behaves like a small, realistic streaming data platform.